NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and IT

ParkWise

Explainable Parking Reservation and Reallocation System

COMPLEX COMPUTING PROBLEM REPORT

CT-351 | Knowledge Representation & Reasoning

A knowledge-driven simulation
34 Horn rule schemas · 10 bays · 3 zones · 16 scenarios
Independent inference engines · atomic booking replacement

Group members and roll numbers: awaiting confirmation

Section: awaiting confirmation

Instructor: awaiting confirmation

Submission date: awaiting confirmation

CLO-3: Design/Analysis | PLO-10: Lifelong Learning

Version 2.1 · Technical build: 10 November 2026

Synthetic educational model. Group identity and actual participation must be completed before submission.

DEPARTMENT OF COMPUTER SCIENCE AND IT

Course Code: CT-351 | Course Title: Knowledge Representation & Reasoning

Complex Computing Problem (CCP) - Grading Rubric

Student No.	Name	Roll No.
S1		
S2		
S3		
S4		

CRITERIA AND SCALES	0 (Unsatisfactory)	1 (Developing)	2 (Proficient)	S1	S2	S3	S4
Criterion 1: Problem Statement & Motivation	Objective of the project and domain requirements are not clear or described.	Some lack of clarity in objectives/purpose or domain justification.	The project's objectives and domain motivation are clearly stated and justified.				
Criterion 2: Structural KR (Frames & Semantic Networks)	Model fails to represent necessary concepts, frames, slots, or semantic relationships.	Model includes frames and semantic networks but lacks complete slots, facets, or default values.	Model meets Proficient criteria with comprehensive, accurate frame structures and semantic networks.				
Criterion 3: Formal Logic (FOL & Description	Logical representations contain major syntax/semantic	FOL and DL formulas are present but contain minor	Model represents domain constraints perfectly using				

Logic)	errors or are missing.	syntax/quantifier errors.	correct FOL formulas and Description Logic axioms.				
Criterion 4: Inference Engine (Horn Clauses & Chaining)	Rules are not converted to Horn clauses; failure to execute chaining.	Rules are in Horn clauses, but Forward/Backward chaining traces have minor logical gaps.	Horn clauses are correctly formulated with flawless, detailed Forward and Backward chaining traces.				
Criterion 5: Team Participation & Individual Viva	The individual did not contribute to the project and failed to answer viva questions.	The individual contributed partially and demonstrated basic understanding during viva.	The individual contributed significantly and demonstrated mastery of KR concepts during viva.				
Total Marks (Out of 10):							

Teacher's Signature: _____ Date: _____

Report contents and abstract

Table 1. Report organization

Section	Coverage	Page
Official rubric	Unmarked teacher-supplied grading sheets	2-3
1. Introduction	Motivation, objectives, scope and system context	5
2. Domain Analysis	Use cases, requirements, entities and relations	6
3. Knowledge Representation	Frames, inheritance, facets and semantic network	10
4. Formal Logic Representation	FOL vocabulary, constraints, finite checks and DL	13
5. Horn Clause Knowledge Base	Complete 34-schema policy and default base facts	18
6. Inference and Reasoning	Forward chaining, backward proofs and failures	22
7. Implementation	Architecture, state transitions, code and simulation	26
8. Results and Discussion	16 cases, test evidence, benchmarks and limitations	33
9. Conclusion	Achievements and realistic next steps	38
10. References	Course template and KR background sources	39
11. Team Participation	Unassigned responsibilities and viva preparation	40
Teacher-brief supplement	Requirement map, script, limitations, full goal trace	41-44
Appendices	Complete core Python implementation and run guide	45

Abstract

ParkWise is an explainable parking reservation simulator for a synthetic mixed-use facility. It combines structural knowledge models with executable finite Horn reasoning to identify suitable bays for timed requests. Eligibility accounts for verification, operating hours, vehicle category and dimensions, accessibility, charging readiness, budget, maximum stay, sensor state and reservation overlap. Two independent inference engines produce candidate decisions and inspectable proofs. A transaction-style replacement action preserves the original booking when reallocation fails. The implementation includes Python and standalone JavaScript versions of the same policy, checked on 616 input snapshots. Forty-four Python tests and 49 DOM interaction assertions passed. The model does not claim live sensing, real account verification or a complete OWL reasoner.

1. Introduction

1.1 Background and motivation

Parking decisions depend on more than whether a bay looks empty. A space may be reserved for part of the requested period, unsuitable for the vehicle height, unavailable because its charger is faulty, or uncertain because its occupancy reading is unknown. A driver also has an intended stay and a budget. An explainable decision should identify which conditions are satisfied and which prevent allocation.

The selected domain is a synthetic multi-zone parking facility. ParkWise models ten bays across Ground, Upper deck and Charging court zones. This setting deliberately differs from a campus access system based on student, employee and visitor permits: its primary decision is interval-aware compatibility and reservation, followed by safe recovery from disruptions.

1.2 Problem statement

Given a request, a complete current booking collection and a facility snapshot, derive every bay that satisfies all hard constraints for the whole interval. Recommend one using a deterministic order, explain its proof and the failed conditions for alternatives, and allow a confirmation only after rechecking current state. If a confirmed bay becomes unavailable, find a replacement without losing the original booking on failure.

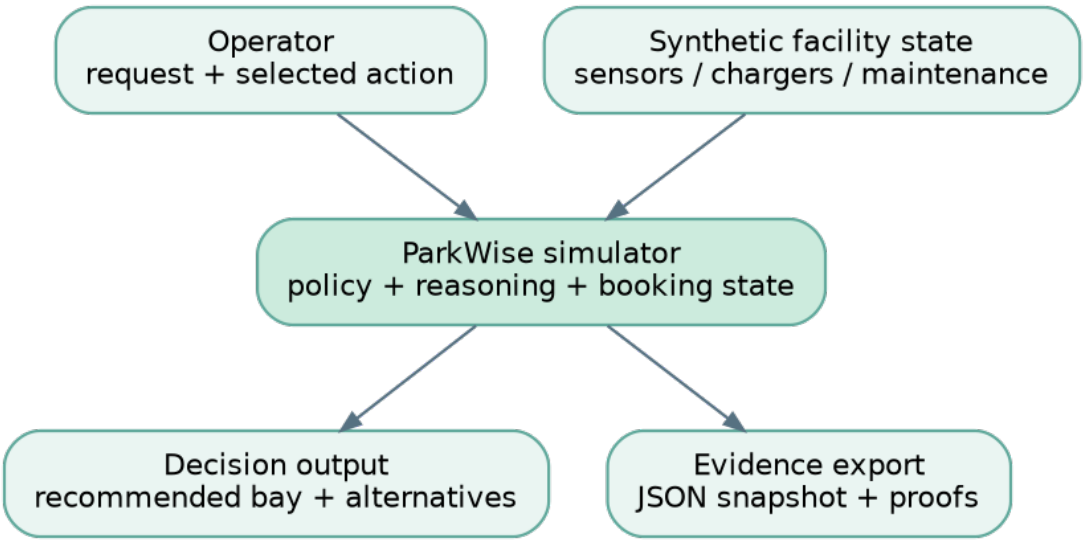


Figure 1. System context. Facility and sensor information is simulated input, not a live integration.

1.3 Objectives

• Represent the domain with executable frame defaults, a semantic network, quantified FOL statements and a DL vocabulary.

• Implement 34 policy schemas with independent forward and backward inference.

• Simulate booking intervals, changing facility conditions and atomic reallocation.

• Validate decisions, integrity conditions and cross-language equivalence using reproducible tests.

2. Domain analysis: actors and use cases

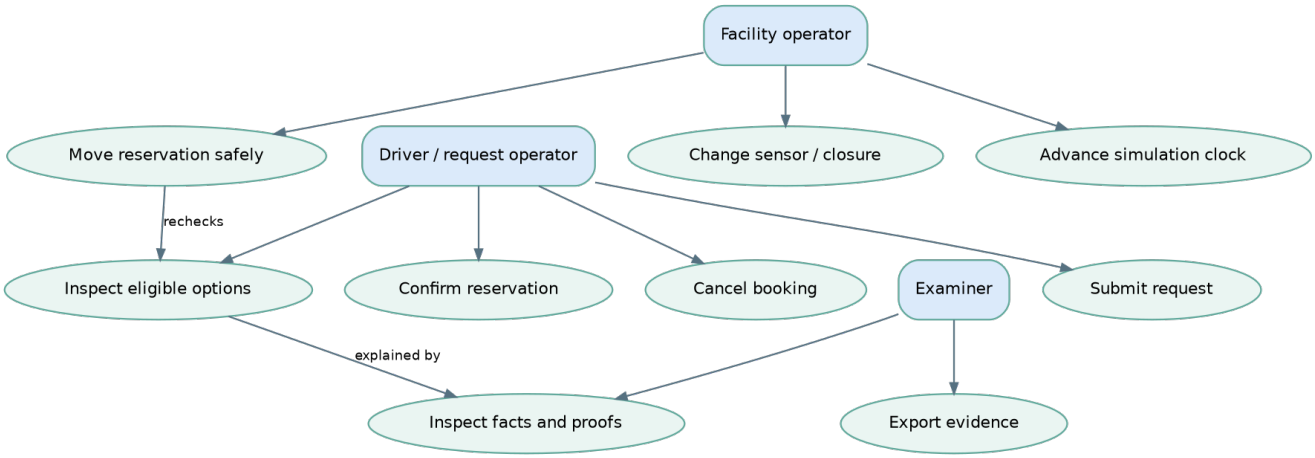


Figure 2. Use-case view showing operator, driver-request and examiner interactions.

2.1 Actors

The driver/request operator enters the vehicle and time requirements and confirms a suggested bay. The facility operator changes synthetic sensor, maintenance or charger states and handles disruptions. The examiner inspects proofs, steps through the inference trace and exports evidence. These are conceptual roles in one local interface, not separate authenticated accounts.

2.2 Use-case boundaries

Confirm, cancel and move change the session booking list. Evaluate and inspect are read-only operations on a snapshot. Advance clock changes the displayed lifecycle classification; it does not pretend to detect a real vehicle arrival. No payment, vehicle gate or external permit database is connected.

The knowledge acquisition method is requirements decomposition: each condition is stated as a project assumption, encoded in facts or rules and paired with a test. No interviews or official parking-policy research are asserted.

2.1 Domain requirements and constraints

Table 2. Requirement-to-implementation mapping

ID	Requirement	Mechanism
DR01	Verified, enabled account	H01, H15, H25, H34
DR02	Valid full interval inside 06:00-22:00	Validation; H02, H26
DR03	Vehicle category matches bay category	H03-H06
DR04	Accessible bay requires badge	H07-H08, H27
DR05	Charging demand requires EV and working charger	Validation; H09-H10, H21, H28
DR06	Height and width must both fit	Adapter; H11, H22
DR07	Estimated fee must fit request budget	Adapter; H12, H23
DR08	Stay must not exceed bay maximum	Adapter; H12, H24
DR09	No overlapping reservation on same bay	Adapter; H14, H17; IC4
DR10	Closed bays cannot accept new requests	H14, H18
DR11	Sensor must explicitly report free	H14, H19-H20
DR12	Choose by distance, then cost, then bay ID	Application ranking
DR13	Recheck before confirmation; unique request ID	commit + validator
DR14	Failed replacement retains original booking	reallocate trial-copy action
DR15	Expose pending, active and completed intervals	H29-H31; simulation clock
DR16	Independent engines agree on candidate truth	evaluate agreement check

The hard constraints are eligibility, interval exclusivity and unique booking identity. Ranking is a preference applied after every hard constraint is satisfied. Unknown occupancy is not treated as free. Completed reservations remain in history but cease to overlap later intervals.

2.2 Facility data and policy assumptions

Table 3. Ten bay instances in the default synthetic facility

Bay	Zone	Type	Walk / rate	Special condition
G1	Ground	general	20 m / 100 PKR/h	General use
G2	Ground	general	40 m / 80 PKR/h	General use
A1	Ground	accessible	5 m / 100 PKR/h	Badge required
E1	Charging court	charging	25 m / 180 PKR/h	Charger ready
M1	Ground	motorcycle	15 m / 40 PKR/h	General use
G3	Upper deck	general	65 m / 50 PKR/h	Height 1.7 m
G4	Ground	general	30 m / 60 PKR/h	Max 30 minutes
E2	Charging court	charging	45 m / 160 PKR/h	Charger fault
A2	Upper deck	accessible	55 m / 90 PKR/h	Badge required
G5	Upper deck	general	50 m / 70 PKR/h	Sensor occupied

Time model

Times are integer minutes since midnight. Intervals are half-open: [start,end). The exact overlap condition is start1 < end2 AND start2 < end1. This handles partial overlap and containment, while allowing one booking to end exactly when another begins. Requests are same-day only.

Price model

```
estimated_cost = ceil(duration_minutes * hourly_rate / 60)
```

The estimated price is rounded upward to a whole PKR. These are synthetic instructional rates, not advertised market prices. The system checks affordability but does not collect payment. Default car dimensions are 1.8 m height and 1.8 m width; the upper-deck G3 bay admits at most 1.7 m height.

Conservative availability

A sensor reading of occupied or unknown blocks a recommendation even for a future interval. This is an intentionally conservative snapshot policy. A future version should model the time validity of sensor readings rather than interpreting them as indefinitely current.

2.2 Entity relationships and data model

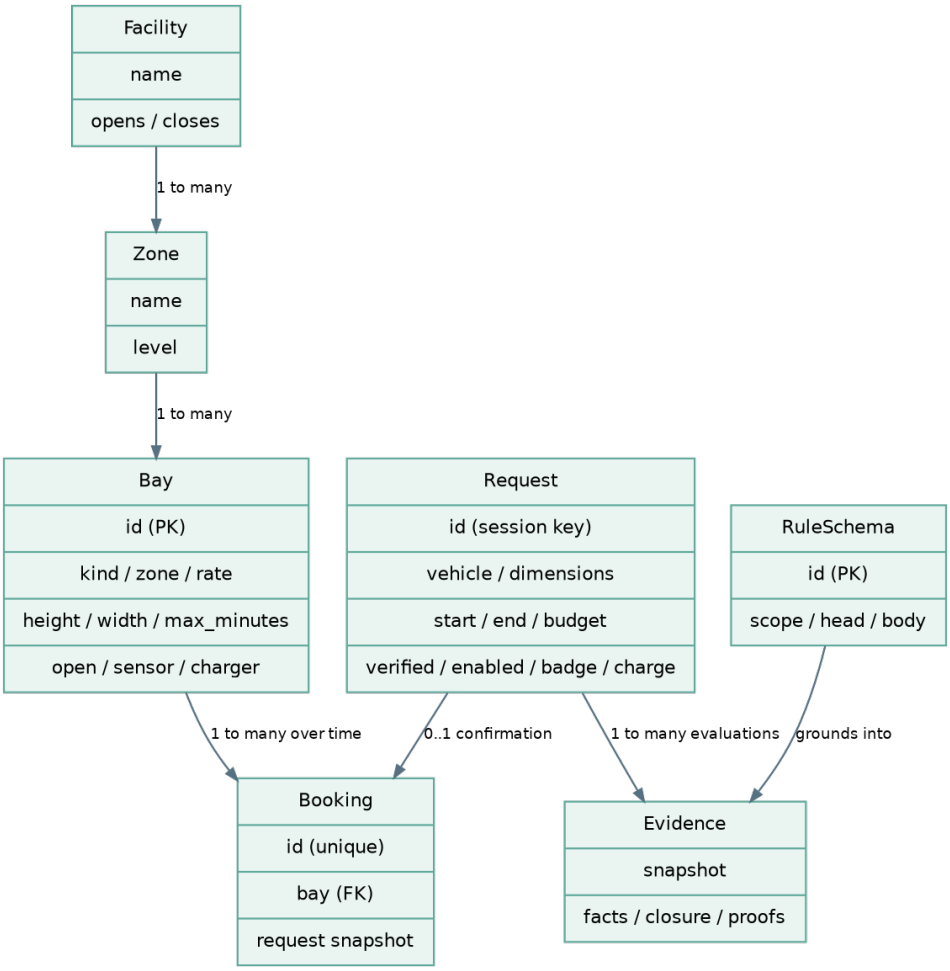


Figure 3. Logical entity relationships. Cardinalities describe the model; runtime storage uses dictionaries and lists.

A facility contains zones and each bay belongs to one zone. A request may become one confirmed booking; a bay may receive many non-overlapping bookings over time. Booking identity is the request ID, unique within the session. A booking stores the original request attributes so later replacement uses the same requirements.

Table 4. Entity-to-runtime mapping

Concept	Executable representation
Bay, sensor and charger	One bay dictionary; sensor and charger states embedded as slots
Request and vehicle	Request dictionary with vehicle category and dimensions
Booking	Request snapshot plus bay reference
Facility / zone	Fixed operating hours; zone slot on each bay
Rule schema / evidence	policy.json; generated facts, trace and proof objects

The conceptual model is richer than the normalized storage. A separate database table for every concept is unnecessary for this local simulator, but the relations are shown explicitly so the representation can be extended without confusing vehicle values with independent vehicle records.

3. Knowledge representation: frame catalogue

Table 5. Principal frames, slots, defaults and facets

Frame	Slots and defaults	Facets
Facility	name; opens=360; closes=1320	Daily interval; synthetic facility
Zone	name; level=0	Unique conceptual zone name
Bay	id; kind=general; open=true; distance=0; rate=100; max_height=2.1; max_width=2.2; max_minutes=480	Unique ID; nonnegative rate/distance; positive limits
GeneralBay / AccessibleBay	Inherit Bay; override kind	Accessible type requires badge via policy
ChargingBay / MotorcycleBay	Inherit Bay; override kind	Service/category restrictions via policy
Request	id; vehicle=car; start=540; end=600; verified=false; enabled=true; badge=false; charge=false; height=1.8; width=1.8; budget=500	Valid same-day interval; positive dimensions; integer budget
Booking	Inherits Request; adds bay	Known bay reference; no overlap
Vehicle / ElectricVehicle / Motorcycle	Vehicle category with subclass override	Category enumerations
Sensor	reading=unknown	free, occupied or unknown
Charger	status=fault	ready or fault

The conservative Request default is unverified. The demonstration constructor explicitly sets verified=true for its sample inputs; this is an instance override, not a logical inference of identity. Likewise the bay constructor supplies known initial sensor/charger values for the synthetic scenario.

Facet descriptions are recorded in frames.json. The runtime validator explicitly enforces the supported facets; it is not a generic interpreter for arbitrary textual facets. Missing data must not be silently converted into authorization.

3.1 Inheritance and default resolution

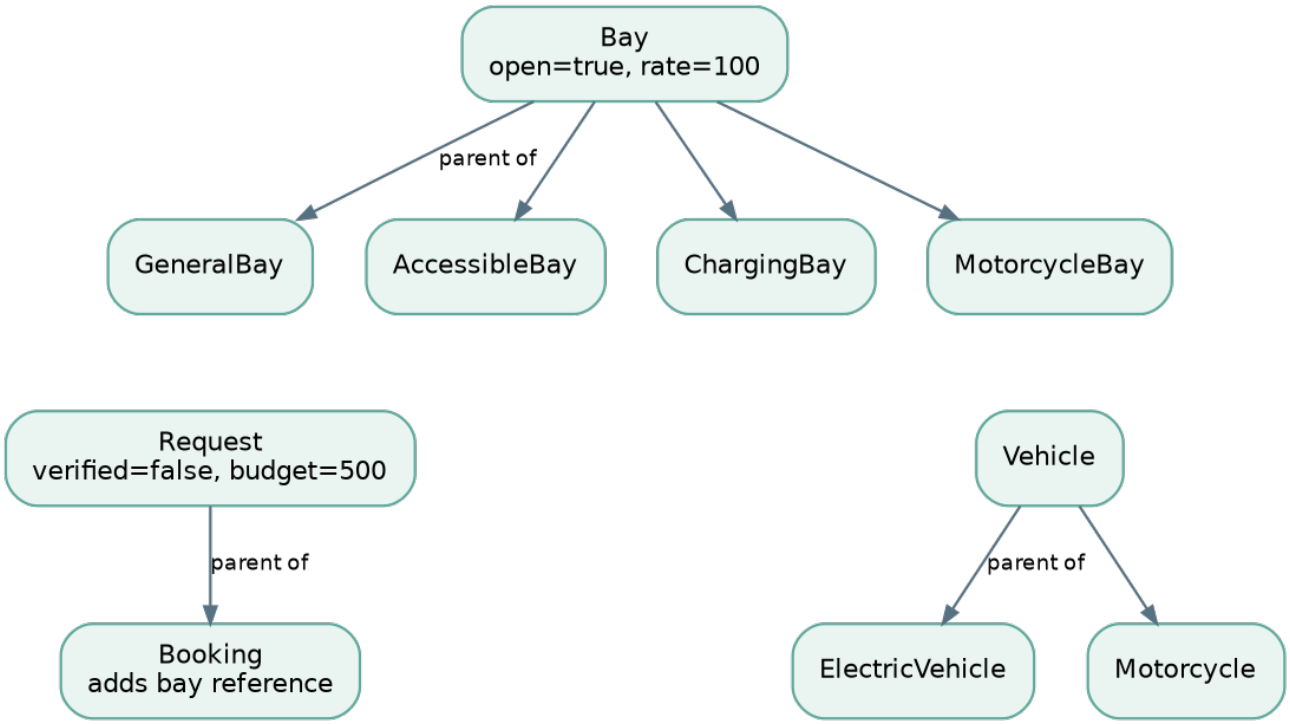


Figure 4. Inheritance among the principal frames. Independent supporting frames are listed in Table 5.

```
def frame_defaults(name,trail=()):
    if name in trail:raise ValueError('Cyclic frame inheritance.')
    f=FRAME_CATALOG[name];base=frame_defaults(f['parent'],trail+(name,)) if 'parent' in f else {}
    return {**base,**f.get('slots',{})}
```

Table 6. Executable default-resolution examples

Expression	Selected actual output
frame_defaults("ChargingBay")	kind=charging; open=True; rate=100; max_height=2.1
frame_defaults("Booking")	Request slots inherited; bay=None added
request()	verified=True explicitly overrides Request default=False

The recursive resolver follows one parent chain, merges slots and detects cycles. Frames therefore contribute actual runtime defaults. Rules still determine eligibility; inheriting a default does not bypass verification, budget or availability checks. The test suite checks inheritance and the matching of the declared values used by the application.

3.2 Semantic network

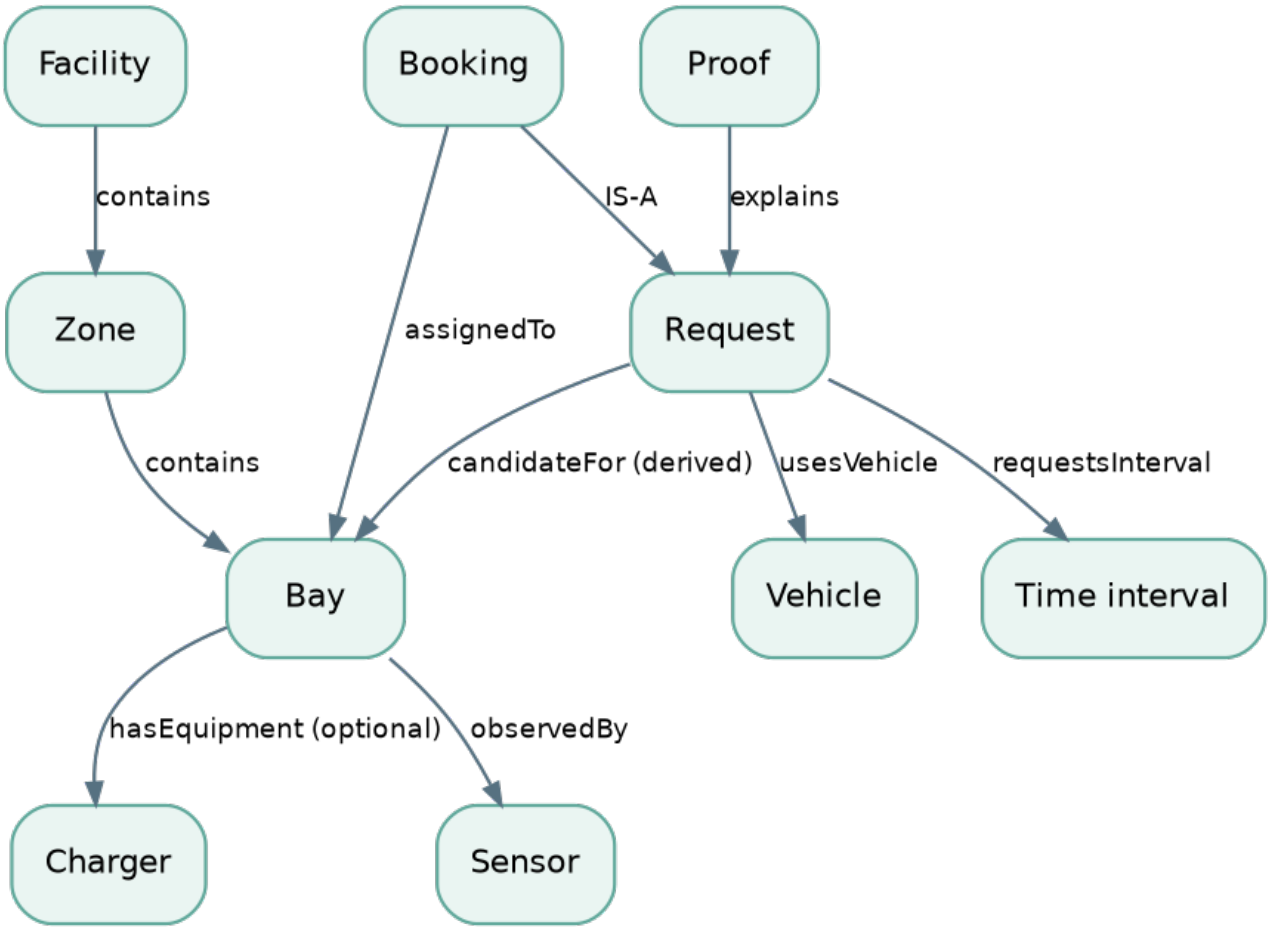


Figure 5. Semantic network with labelled ownership, containment, inheritance and derived relations.

Interpretation of important links

• Booking IS-A Request: a confirmed record retains the request requirements.

• Bay observedBy Sensor and hasEquipment Charger: operational evidence affects availability and charging service.

• Request candidateFor Bay: a derived relation, unlike the stored assignedTo relation of a booking.

• Proof explains Request: the evidence records why conditions succeeded or failed for the evaluated snapshot.

The semantic network communicates relationships; it does not compute a shortest route or interval arithmetic. Walking distance is an input attribute. The proof tree is generated by inference and is distinct from this conceptual network.

4.1 First-order logic: vocabulary

Table 7. Predicate groups and meanings

Group	Predicates and interpretation
Request facts	request(r), verified(r), unverified(r), account_enabled(r), account_disabled(r)
Schedule facts	valid_window(r), inside_hours(r), outside_hours(r)
Vehicle / service	car(r), ev(r), motorcycle(r), badge(r), no_badge(r), charge(r), no_charge(r)
Bay category	bay(b), car_space(b), motorcycle_space(b), accessible(b), unrestricted(b), charging(b), noncharging(b)
Facility state	operational(b), closed(b), sensor_free(b), sensor_occupied(b), sensor_unknown(b), charger_ready(b), charger_fault(b)
Arithmetic / interval facts	height_ok(r,b), width_ok(r,b), budget_ok(r,b), duration_ok(r,b), booking_clear(r,b), booking_conflict(r,b)
Intermediate conclusions	authorized(r), schedulable(r), car_sized(r), size_fit(r,b), access_fit(r,b), service_fit(r,b), dimension_fit(r,b), commercial_fit(r,b)
Final eligibility	compatible(r,b), available(r,b), admissible(r), candidate(r,b)
Diagnostic conclusions	blocked(r,b), manual_review(r,b), unaffordable(r,b), duration_rejected(r,b), access_rejected(r,b), service_rejected(r,b), identity_rejected(r), schedule_rejected(r)
Lifecycle	booking_future(q), booking_current(q), booking_past(q); reservation_pending(q), reservation_active(q), reservation_completed(q)

r and q range over request/booking identifiers; b ranges over bay identifiers. Time and measured attributes range over numeric values. Constants include R1, G1, E1 and the other listed instance IDs. Lowercase predicate names match the executable representation. Positive complementary facts are generated explicitly from validated input; neither engine infers negation merely from an absent fact.

4.1 Quantified rules and constraints

The following notation writes quantifiers explicitly: FORALL means universal quantification and EXISTS means existential quantification. AND, OR, NOT and -> retain their usual logical meanings. Start, End, Height, Cost and BayOf are attribute functions in the specification; the program evaluates them from dictionaries.

```
F01 FORALL r: verified(r) AND account_enabled(r) -> authorized(r)
F02 FORALL r: request(r) AND valid_window(r) AND inside_hours(r)
    -> schedulable(r)
F03 FORALL r,b: size_fit(r,b) AND access_fit(r,b)
    AND service_fit(r,b) AND dimension_fit(r,b) -> compatible(r,b)
F04 FORALL r,b: bay(b) AND operational(b) AND sensor_free(b)
    AND booking_clear(r,b) -> available(r,b)
F05 FORALL r,b: admissible(r) AND compatible(r,b)
    AND available(r,b) AND commercial_fit(r,b) -> candidate(r,b)
F06 FORALL r,q: Overlap(r,q) <->
    Start(r)<End(q) AND Start(q)<End(r)
F07 FORALL r,b: booking_clear(r,b) <->
    NOT EXISTS q: Booked(q,b) AND Overlap(r,q)
F08 FORALL r,q,b: Booked(r,b) AND Booked(q,b) AND r!=q
    -> NOT Overlap(r,q)
F09 FORALL r,b,c: Booked(r,b) AND Booked(r,c) -> b=c
F10 FORALL r,b: Booked(r,b) -> bay(b)
F11 FORALL r: BookedRequest(r) -> 360<=Start(r)<End(r)<=1320
F12 FORALL r: charge(r) -> ev(r)
F13 FORALL r,b: candidate(r,b) -> Cost(r,b)<=Budget(r)
F14 FORALL r,b: candidate(r,b) -> Height(r)<=MaxHeight(b)
    AND Width(r)<=MaxWidth(b)
F15 FORALL b: sensor_free(b) OR sensor_occupied(b)
    OR sensor_unknown(b)
F16 FORALL b: NOT(sensor_free(b) AND sensor_unknown(b))
```

F01-F05 are representative rule readings; Section 5 gives every executable rule schema. F06-F07 specify arithmetic and complete-list adapter semantics. F08-F12 and the typed input validator protect stored records. F13-F14 follow from the candidate rule and adapter checks. F15-F16 describe the validated sensor enumeration; the remaining pairwise exclusions are analogous.

The formulas do not imply that every candidate is booked. Selection and confirmation are separate application actions. A closure after confirmation also does not make the historical booking disappear.

4.1 Finite-model checks and interval examples

Table 8. Executable integrity checks in theory.py

Check	Property	Enforcement / evidence
IC1	Unique bay IDs	Validator; finite check
IC2	Unique booking IDs	Validator; finite check
IC3	Every booking references a known bay	Validator; malformed-reference test
IC4	No overlapping bookings on one bay	Validator; injected-conflict test
IC5	Valid same-day intervals	Validator; finite check
IC6	Booked intervals fit operating hours	Eligibility before commit; finite check
IC7	Charging request implies EV category	Validator; finite check
IC8	Booking snapshot verified and enabled	H01/H15 before commit; finite check

Worked overlap cases

Table 9. Interval arithmetic with half-open boundaries

Existing booking	New request	Overlap?	Effect on that bay
09:00-10:00	09:30-10:30	Yes	Excluded
09:00-10:00	09:10-09:20	Yes	Excluded (containment)
09:00-10:00	08:30-10:30	Yes	Excluded (enclosing)
09:00-10:00	10:00-11:00	No	May be reused
09:00-10:00	08:00-09:00	No	May be reused

For the overlapping case, $540 < 630$ and $570 < 600$ are both true. For the adjacent case, $600 < 600$ is false. The finite checker deliberately reports a counterexample when two manually injected bookings occupy G1 during the same interval. This tests rejection of inconsistent input rather than assuming all inputs are correct.

These checks evaluate one finite state. They are not a general theorem prover and do not establish that arbitrary FOL formulas hold in every possible interpretation.

4.2 Description Logic: TBox and ABox

The DL specification captures concept subsumption, role restrictions, membership and disjointness. In the notation below, SUB denotes subsumption, SOME denotes existential restriction, AND denotes concept intersection, and BOTTOM is the empty concept. These are readable renderings of standard DL constructs [2].

```
GeneralBay SUB Bay       AccessibleBay SUB Bay
ChargingBay SUB Bay      MotorcycleBay SUB Bay
ElectricVehicle SUB Vehicle     Motorcycle SUB Vehicle
Car SUB Vehicle
Booking SUB Request
Booking SUB SOME forBay.Bay
Bay SUB SOME partOf.Facility
ChargingRequest SUB Request
ChargingRequest SUB SOME usesVehicle.ElectricVehicle
GeneralBay AND AccessibleBay SUB BOTTOM
GeneralBay AND ChargingBay SUB BOTTOM
GeneralBay AND MotorcycleBay SUB BOTTOM
AccessibleBay AND ChargingBay SUB BOTTOM
AccessibleBay AND MotorcycleBay SUB BOTTOM
ChargingBay AND MotorcycleBay SUB BOTTOM
ElectricVehicle AND Motorcycle SUB BOTTOM
```

ABox examples and role assertions

```
Facility(lot)
GeneralBay(G1), GeneralBay(G2), AccessibleBay(A1)
ChargingBay(E1), MotorcycleBay(M1)
partOf(G1,lot), partOf(G2,lot), partOf(A1,lot)
partOf(E1,lot), partOf(M1,lot)
```

The complete ontology.ttl also declares all ten bay individuals, zone concepts and the object-property vocabulary. The executable taxonomy component supports only named-class superclass closure and declared disjointness checks. It does not claim existential completion, universal restrictions, cardinality reasoning or full OWL consistency checking.

Open-world distinction

The absence of an OWL assertion does not make it false. The reservation adapter instead uses a complete in-session booking collection to establish booking_clear. That closed collection assumption is explicit and does not change the meaning of OWL assertions.

4.2 Classification and inconsistency example

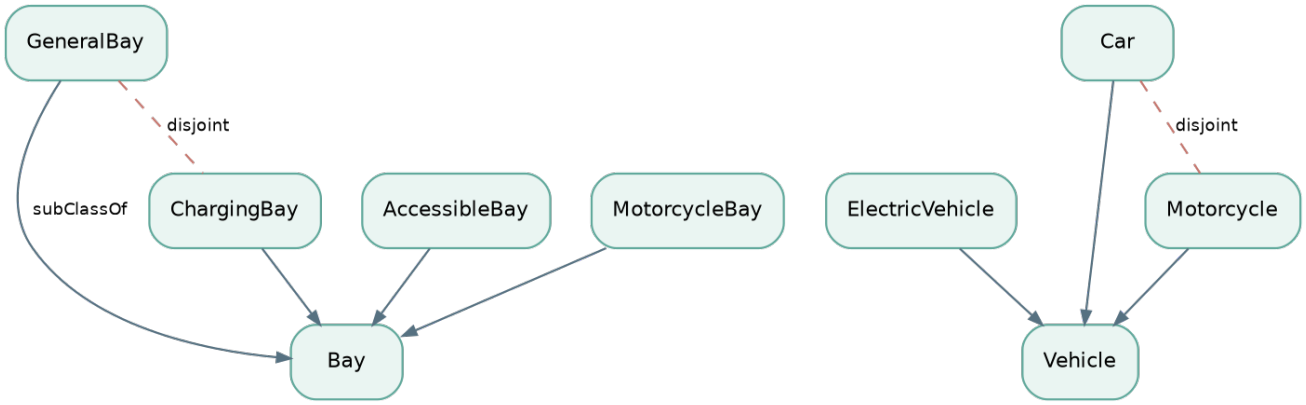


Figure 6. Selected taxonomy relationships. Dashed links show examples of declared disjointness.

Table 10. Actual taxonomy classifier output

Input	Superclass closure / violation
G1: GeneralBay	G1: Bay, GeneralBay
E1: ChargingBay	E1: Bay, ChargingBay
bad: GeneralBay, ChargingBay	bad: Bay, GeneralBay, ChargingBay; disjointness violation

The classifier repeatedly adds named superclasses until no new class is obtained. It then checks whether an instance contains both members of a disjoint pair. This operation is independently tested for positive classification, a consistent ABox and an injected inconsistency.

```
types = classify({"G1": ["GeneralBay"],
                  "bad": ["GeneralBay", "ChargingBay"]})
# G1 inherits Bay.
# bad is reported against the GeneralBay/ChargingBay disjoint pair.
```

This component strengthens the implementation without claiming that a hand-written class closure algorithm is equivalent to a complete DL reasoner. The report distinguishes the richer ontology specification from the subset executed by theory.py.

5. Horn knowledge base: eligibility rules

All variables in a schema are universally quantified. The finite grounder substitutes the current request for {r}, each bay for {b}, or each stored booking for {q}, according to the scope. A conjunction in the body implies its single positive head.

H01 · Verified and enabled account

```
authorized({r}) :-
    verified({r}), account_enabled({r}).
```

H02 · Whole interval inside facility hours

```
schedulable({r}) :-
    request({r}), valid_window({r}), inside_hours({r}).
```

H03 · Car body category

```
car_sized({r}) :-
    car({r}).
```

H04 · EV body category

```
car_sized({r}) :-
    ev({r}).
```

H05 · Car-sized vehicle fits car-space category

```
size_fit({r},{b}) :-
    car_sized({r}), car_space({b}).
```

H06 · Motorcycle category match

```
size_fit({r},{b}) :-
    motorcycle({r}), motorcycle_space({b}).
```

H07 · Unrestricted bay access

```
access_fit({r},{b}) :-
    request({r}), unrestricted({b}).
```

H08 · Badge permits accessible bay

```
access_fit({r},{b}) :-
    badge({r}), accessible({b}).
```

H09 · Non-charging request uses non-charging bay

```
service_fit({r},{b}) :-
    no_charge({r}), noncharging({b}).
```

H10 · Charging requires compatible working equipment

```
service_fit({r},{b}) :-
    charge({r}), ev({r}), charging({b}), charger_ready({b}).
```

H11 · Both physical dimensions fit

```
dimension_fit({r},{b}) :-
    height_ok({r},{b}), width_ok({r},{b}).
```

H12 · Price estimate and maximum stay accepted

```
commercial_fit({r},{b}) :-
    budget_ok({r},{b}), duration_ok({r},{b}).
```

5. Horn rules: candidates and diagnostics

All variables in a schema are universally quantified. The finite grounder substitutes the current request for {r}, each bay for {b}, or each stored booking for {q}, according to the scope. A conjunction in the body implies its single positive head.

H13 · All compatibility conditions hold

```
compatible({r},{b}) :-
    size_fit({r},{b}), access_fit({r},{b}), service_fit({r},{b}), dimension_fit({r},{b}).
```

H14 · Known free and no interval conflict

```
available({r},{b}) :-
    bay({b}), operational({b}), sensor_free({b}), booking_clear({r},{b}).
```

H15 · Account and schedule accepted

```
admissible({r}) :-
    authorized({r}), schedulable({r}).
```

H16 · Eligible booking option

```
candidate({r},{b}) :-
    admissible({r}), compatible({r},{b}), available({r},{b}), commercial_fit({r},{b}).
```

H17 · Overlapping reservation blocks bay

```
blocked({r},{b}) :-
    booking_conflict({r},{b}).
```

H18 · Maintenance closure blocks bay

```
blocked({r},{b}) :-
    request({r}), closed({b}).
```

H19 · Occupied sensor reading blocks bay

```
blocked({r},{b}) :-
    request({r}), sensor_occupied({b}).
```

H20 · Uncertain occupancy needs review

```
manual_review({r},{b}) :-
    request({r}), sensor_unknown({b}).
```

H21 · Charging hardware fault blocks charging request

```
blocked({r},{b}) :-
    charge({r}), charger_fault({b}).
```

H22 · Vehicle clearance exceeded

```
blocked({r},{b}) :-
    dimension_exceeded({r},{b}).
```

H23 · Estimated price exceeds request budget

```
unaffordable({r},{b}) :-
    over_budget({r},{b}).
```

H24 · Stay exceeds bay maximum

```
duration_rejected({r},{b}) :-
    over_duration({r},{b}).
```

5. Horn knowledge base: rejection and lifecycle

All variables in a schema are universally quantified. The finite grounder substitutes the current request for {r}, each bay for {b}, or each stored booking for {q}, according to the scope. A conjunction in the body implies its single positive head.

H25 · Unverified request cannot be admitted

```
identity_rejected({r}) :-
    unverified({r}).
```

H26 · Request outside operating hours

```
schedule_rejected({r}) :-
    outside_hours({r}).
```

H27 · Accessible bay needs badge

```
access_rejected({r},{b}) :-
    no_badge({r}), accessible({b}).
```

H28 · Requested charging service absent

```
service_rejected({r},{b}) :-
    charge({r}), noncharging({b}).
```

H29 · Confirmed booking has not started

```
reservation_pending({q}) :-
    booking_future({q}).
```

H30 · Clock lies inside booking interval

```
reservation_active({q}) :-
    booking_current({q}).
```

H31 · Booking interval has ended

```
reservation_completed({q}) :-
    booking_past({q}).
```

H32 · Eligible accessible option

```
accessible_option({r},{b}) :-
    candidate({r},{b}), accessible({b}), badge({r}).
```

H33 · Eligible charging option

```
charging_option({r},{b}) :-
    candidate({r},{b}), charging({b}).
```

H34 · Disabled account cannot be admitted

```
identity_rejected({r}) :-
    account_disabled({r}).
```

There are 34 schemas: 8 request-scoped, 23 bay-scoped and 3 booking-scoped. With ten bays and no bookings, this produces 8 + 23×10 = 238 ground rules. Each additional confirmed booking contributes three lifecycle ground rules. Arithmetic and interval checks supply base facts; they are not hidden negated literals inside the Horn bodies.

5. Complete default base facts

The following 120 facts are the complete base set for the default R1 request at 09:00, before any reservation is confirmed. The complete finite KB consists of these facts and the grounded instances of all schemas in Section 5. The generated horn_kb.txt includes all 238 ground rules explicitly.

```
accessible(A1).
accessible(A2).
account_enabled(R1).
bay(A1).
bay(A2).
bay(E1).
bay(E2).
bay(G1).
bay(G2).
bay(G3).
bay(G4).
bay(G5).
bay(M1).
booking_clear(R1,A1).
booking_clear(R1,A2).
booking_clear(R1,E1).
booking_clear(R1,E2).
booking_clear(R1,G1).
booking_clear(R1,G2).
booking_clear(R1,G3).
booking_clear(R1,G4).
booking_clear(R1,G5).
booking_clear(R1,M1).
budget_ok(R1,A1).
budget_ok(R1,A2).
budget_ok(R1,E1).
budget_ok(R1,E2).
budget_ok(R1,G1).
budget_ok(R1,G2).
budget_ok(R1,G3).
budget_ok(R1,G4).
budget_ok(R1,G5).
budget_ok(R1,M1).
car(R1).
car_space(A1).
car_space(A2).
car_space(E1).
car_space(E2).
car_space(G1).
car_space(G2).
car_space(G3).
car_space(G4).
car_space(G5).
charger_fault(E2).
charger_ready(E1).
charging(E1).
charging(E2).
dimension_exceeded(R1,G3).
duration_ok(R1,A1).
duration_ok(R1,A2).
duration_ok(R1,E1).
duration_ok(R1,E2).
duration_ok(R1,G1).
duration_ok(R1,G2).
duration_ok(R1,G3).
duration_ok(R1,G5).
duration_ok(R1,M1).
height_ok(R1,A1).
height_ok(R1,A2).
height_ok(R1,E1).
height_ok(R1,E2).
height_ok(R1,G1).
height_ok(R1,G2).
height_ok(R1,G4).
height_ok(R1,G5).
height_ok(R1,M1).
inside_hours(R1).
motorcycle_space(M1).
no_badge(R1).
no_charge(R1).
noncharging(A1).
noncharging(A2).
noncharging(G1).
noncharging(G2).
noncharging(G3).
noncharging(G4).
noncharging(G5).
noncharging(M1).
operational(A1).
operational(A2).
operational(E1).
operational(E2).
operational(G1).
operational(G2).
operational(G3).
operational(G4).
operational(G5).
operational(M1).
over_duration(R1,G4).
request(R1).
sensor_free(A1).
sensor_free(A2).
sensor_free(E1).
sensor_free(E2).
sensor_free(G1).
sensor_free(G2).
sensor_free(G3).
sensor_free(G4).
sensor_free(M1).
sensor_occupied(G5).
unrestricted(E1).
unrestricted(E2).
unrestricted(G1).
unrestricted(G2).
unrestricted(G3).
unrestricted(G4).
unrestricted(G5).
unrestricted(M1).
valid_window(R1).
verified(R1).
width_ok(R1,A1).
width_ok(R1,A2).
width_ok(R1,E1).
width_ok(R1,E2).
width_ok(R1,G1).
width_ok(R1,G2).
width_ok(R1,G3).
width_ok(R1,G4).
width_ok(R1,G5).
width_ok(R1,M1).
```

Clause interpretation

H16 is equivalent to NOT admissible(r) OR NOT compatible(r,b) OR NOT available(r,b) OR NOT commercial_fit(r,b) OR candidate(r,b). Exactly one literal is positive, so it is a definite Horn clause [3]. Diagnostic conclusions such as manual_review are also positive atoms; they are not classical negation of candidate.

6.1 Forward chaining: dependency structure

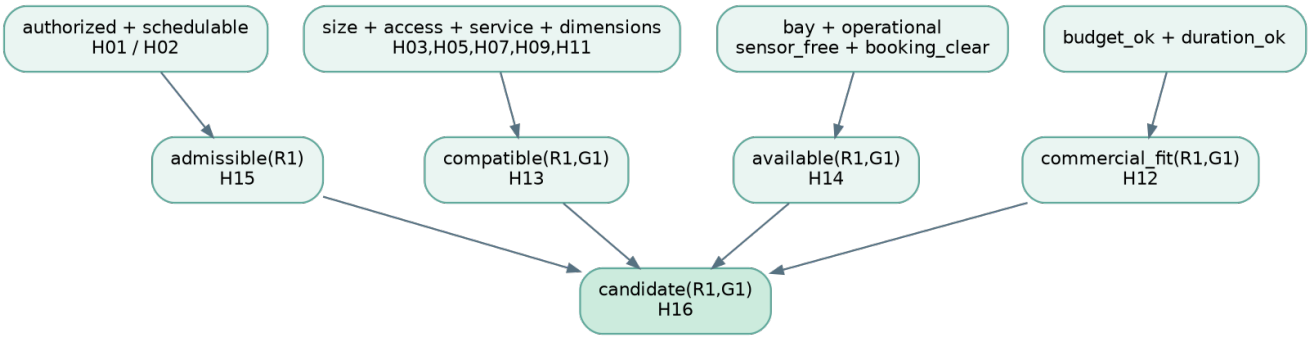


Figure 7. Dependency structure for the default G1 candidate. Arrows point from premises toward conclusions.

Algorithm

The indexed engine builds a reverse map from each premise to the rules waiting for it. A remaining-premise counter is stored for every rule. Initial facts enter an agenda; when an agenda fact is processed, only its dependent rules are updated. A rule fires when its counter reaches zero, and a newly derived head is enqueued once.

The knowledge base is finite and positive. No rule creates a new constant, and each head is inserted at most once. Consequently the agenda terminates. Duplicate body atoms are normalized to a set, avoiding double-counting; an empty-body rule is handled as an unconditional fact.

Within a snapshot, conclusions are monotonic. Changes such as cancellation do not retract facts from a running closure; the next evaluation builds a new snapshot. The unoptimized scan implementation is retained in validate_all.py as an independent comparison baseline.

6.1 Worked forward trace: normal arrival

Table 11. Actual dependency trace filtered to the G1 proof

Global step	Rule	New fact	Supporting premises
1	H03	car_sized(R1)	car(R1)
7	H12	commercial_fit(R1,G1)	budget_ok(R1,G1), duration_ok(R1,G1)
16	H09	service_fit(R1,G1)	no_charge(R1), noncharging(G1)
27	H14	available(R1,G1)	bay(G1), operational(G1), sensor_free(G1), booking_clear(R1,G1)
35	H07	access_fit(R1,G1)	request(R1), unrestricted(G1)
41	H02	schedulable(R1)	request(R1), valid_window(R1), inside_hours(R1)
42	H01	authorized(R1)	verified(R1), account_enabled(R1)
47	H11	dimension_fit(R1,G1)	height_ok(R1,G1), width_ok(R1,G1)
52	H05	size_fit(R1,G1)	car_sized(R1), car_space(G1)
61	H15	admissible(R1)	authorized(R1), schedulable(R1)
62	H13	compatible(R1,G1)	size_fit(R1,G1), access_fit(R1,G1), service_fit(R1,G1), dimension_fit(R1,G1)
66	H16	candidate(R1,G1)	admissible(R1), compatible(R1,G1), available(R1,G1), commercial_fit(R1,G1)

Global step numbers refer to the full trace, which also derives conclusions about the other nine bays. The view above is a proof-relevant subset, not a claim that the engine processed only G1. The interface and knowledge.json contain every firing and its premises.

Actual default decision
Base facts: 120; ground rules: 238; new facts: 67. Candidates: G1, G2. Recommended: G1.

Both G1 and G2 satisfy all hard constraints. G1 wins because its 20 m walking distance is less than G2's 40 m, even though G2 has a lower estimated fee. Budget is a hard ceiling, while distance is the first ranking preference. The ranking order is explicit and does not claim to find a globally optimal allocation for future arrivals.

6.2 Backward chaining: successful proof

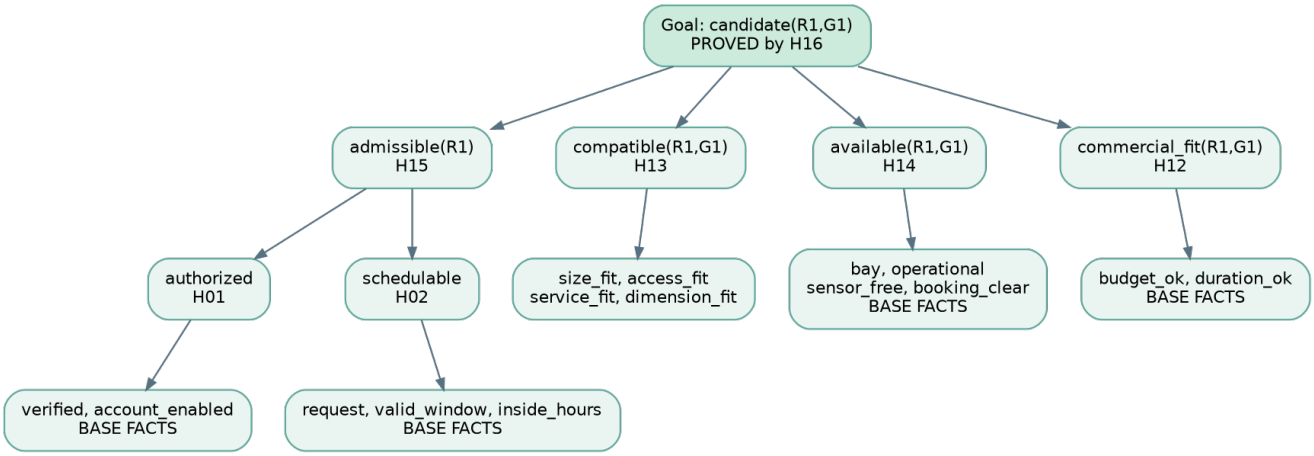


Figure 8. Goal-directed proof structure for candidate(R1,G1). The compatibility branch expands into its four subgoals.

Backward chaining starts with the requested candidate goal, retrieves rules whose head matches it and recursively proves every body atom. Base facts are leaves. If one rule fails, the engine can try another matching rule; for example car_sized has separate car and EV rules. A path guard prevents cyclic recursion.

How this differs from forward chaining

The backward engine is supplied with the original base facts and grounded rules, not the forward closure. Thus its success is independently obtained. The runtime compares the final truth of every candidate goal with forward membership. A head index avoids scanning every rule for each recursive subgoal.

```
goal = "candidate(R1,G1)"
proof = backward(goal, base_facts, grounded_rules)
assert proof["proved"] is True
# UI recursively renders goal, source rule and child proofs.
```

The implementation intentionally avoids global success/failure memoization for arbitrary recursive goals. A naive cached failure can be unsound when a proof was interrupted by a path-specific cycle. For this acyclic eligibility policy, the head index and finite structure keep the proof manageable.

6.2 Failed proof and diagnostic reasoning

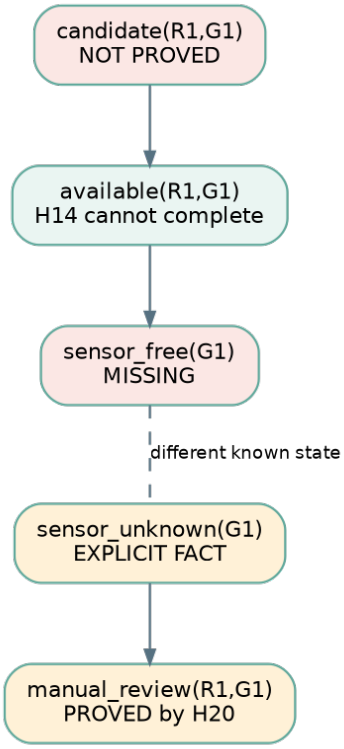


Figure 9. Unknown sensor scenario: missing sensor_free prevents availability; explicit sensor_unknown derives manual_review.

Worked denial: G1 with unknown occupancy

The scenario sets G1.sensor to unknown. The adapter asserts sensor_unknown(G1), not sensor_free(G1). H14 therefore cannot establish available(R1,G1), so H16 cannot prove candidate(R1,G1). Independently, H20 derives manual_review(R1,G1). G2 remains a candidate and is recommended.

Why-not explanations

The interface provides two complementary views. The backward tree identifies the exact unproved subgoal and attempted rule. The decision list translates failed adapter checks into readable categories such as Budget, Maximum stay or Sensor status. These are policy explanations, not probabilities or confidence scores.

Table 12. Different reasons for a missing candidate

Condition	Explicit evidence	Derived diagnostic
Unknown occupancy	sensor_unknown(G1)	manual_review(R1,G1)
Occupied bay	sensor_occupied(G5)	blocked(R1,G5)
Cost exceeds budget	over_budget(R1,G1)	unaffordable(R1,G1)
Charger failure for EV demand	charger_fault(E1), charge(R1)	blocked(R1,E1)

7. Implementation architecture

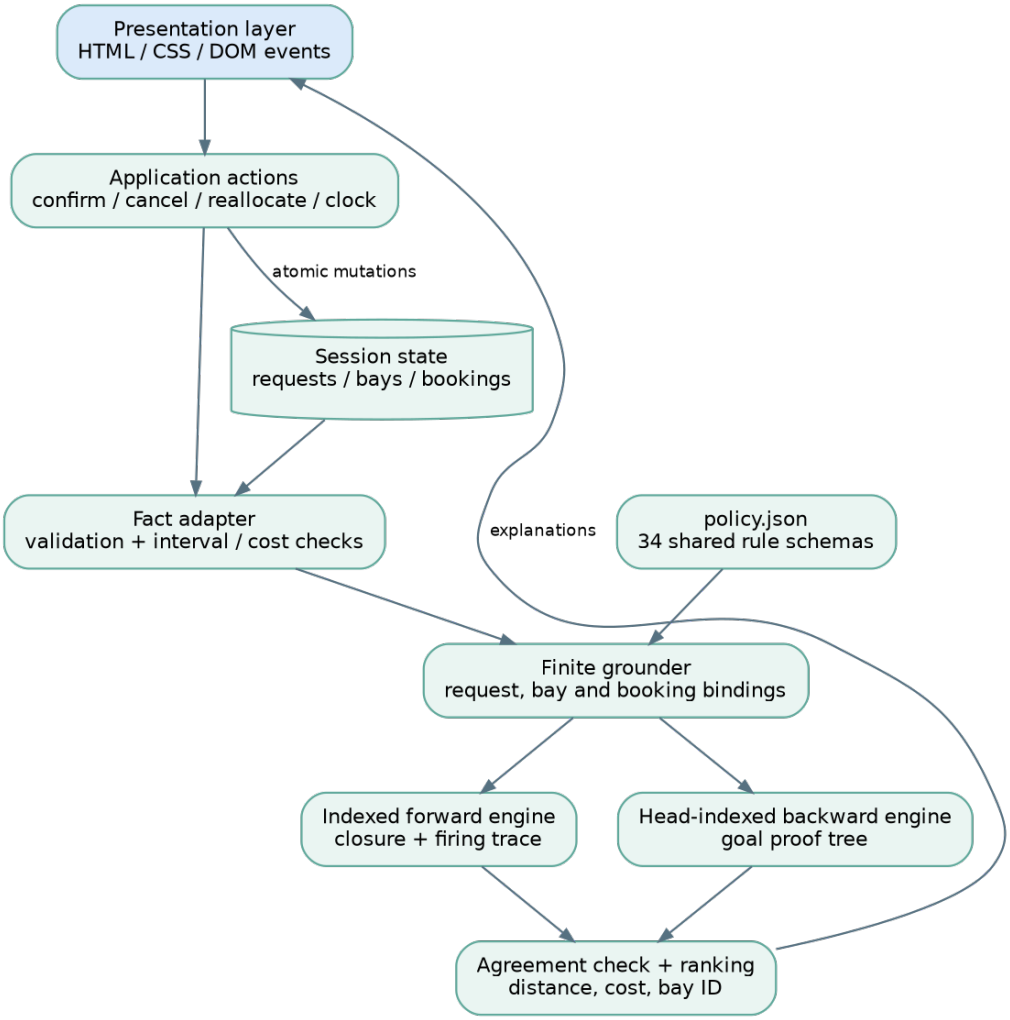


Figure 10. Layered implementation with one shared policy and two independent inference strategies.

Python provides the reference implementation, tests and optional local HTTP server. The standalone HTML file embeds the shared policy, frame catalogue, fixture data, JavaScript engine and interface. It opens directly in a desktop browser without Python or an internet connection. The two engines implement the same adapter and policy semantics and are compared on 616 snapshots.

The offline interface runs its own JavaScript session; it is not synchronized with the optional Python HTTP API. This separation is explicit. The API remains useful for code demonstrations and integration tests, while the single-file simulation removes installation friction for the viva.

There is no persistence service. Export evidence saves the current state, last evaluated snapshot, proofs and event log to a JSON download. Reloading the simulation starts a fresh session. No user information is sent to a server by the offline version.

7.1 Decision workflow and data flow

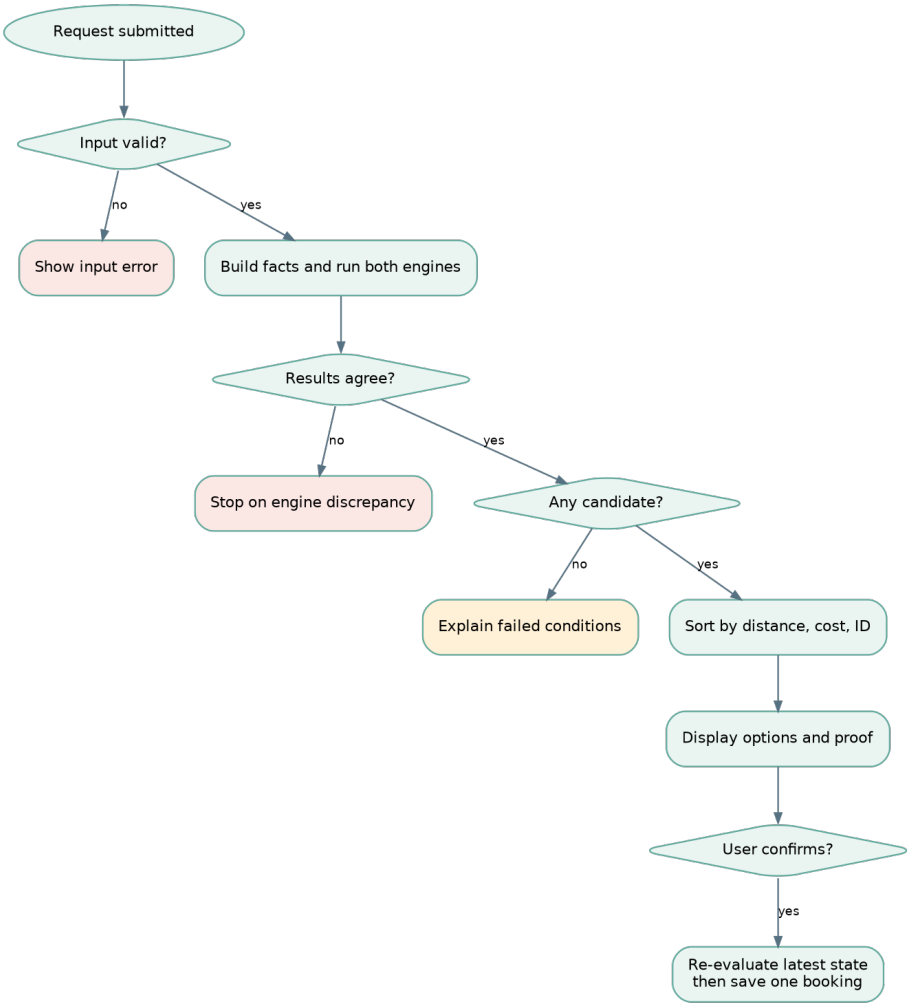


Figure 11. Decision and confirmation workflow. The commit action rechecks the current snapshot before saving.

Single-request decision path

Validation rejects malformed types, IDs, intervals and dimensions before inference. The adapter then performs numeric and interval checks, the finite grounder produces executable clauses and both engines run. Only proved candidates enter ranking. If none exist, the interface displays failed conditions rather than guessing a bay.

Confirm always evaluates again. It does not trust a stale recommendation retained on screen. The browser event loop and Python server process actions sequentially within their respective sessions. This local guarantee is not a claim of multi-user database concurrency safety.

7.1 Booking lifecycle and simulation clock

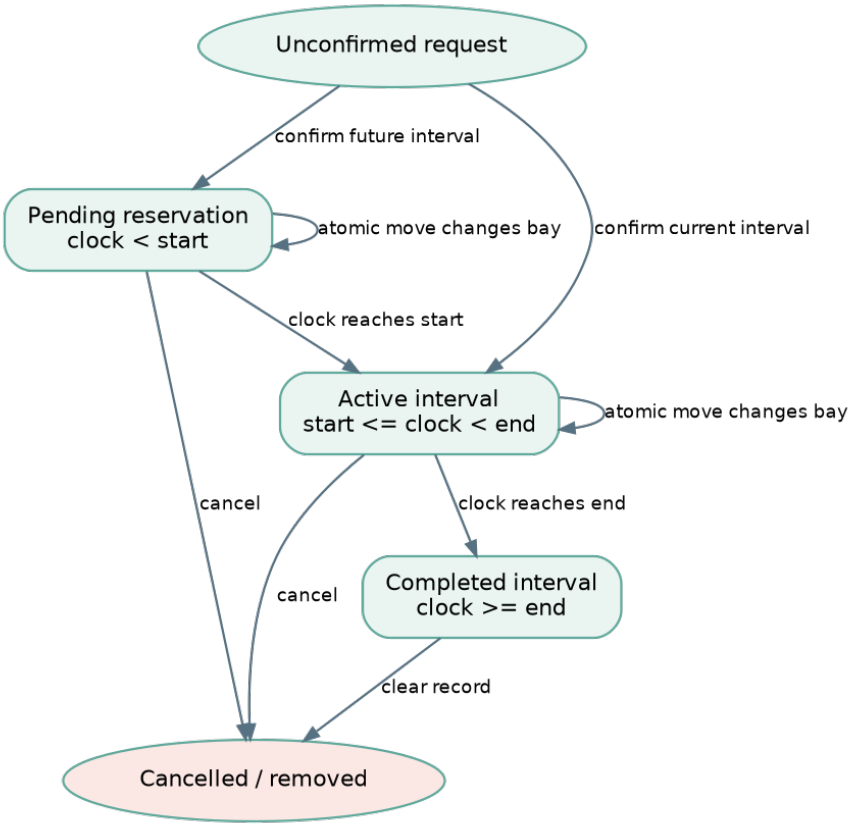


Figure 12. Time-based reservation lifecycle. These are simulated interval states, not detected physical arrivals.

A reservation is pending when the clock is earlier than its start, active while start <= clock < end, and completed when the clock reaches its end. The adapter supplies one lifecycle base fact, and H29-H31 derive the corresponding conclusion. The interface timeline marks the clock in orange and shows each confirmed interval on its assigned bay.

The clock can move forward or backward for demonstration. It does not delete completed records or change their interval. A completed record no longer blocks a later non-overlapping request, because overlap is calculated from the booking times rather than its displayed lifecycle label.

Physical and logical occupancy

The map distinguishes a current confirmed interval from a simulated sensor reading. A bay can be marked occupied because a booking is active or because its sensor is occupied. Candidate eligibility additionally considers the entire requested interval and the sensor policy. Thus a future reservation and a current car are not treated as the same concept.

7.1 Atomic reallocation and recovery

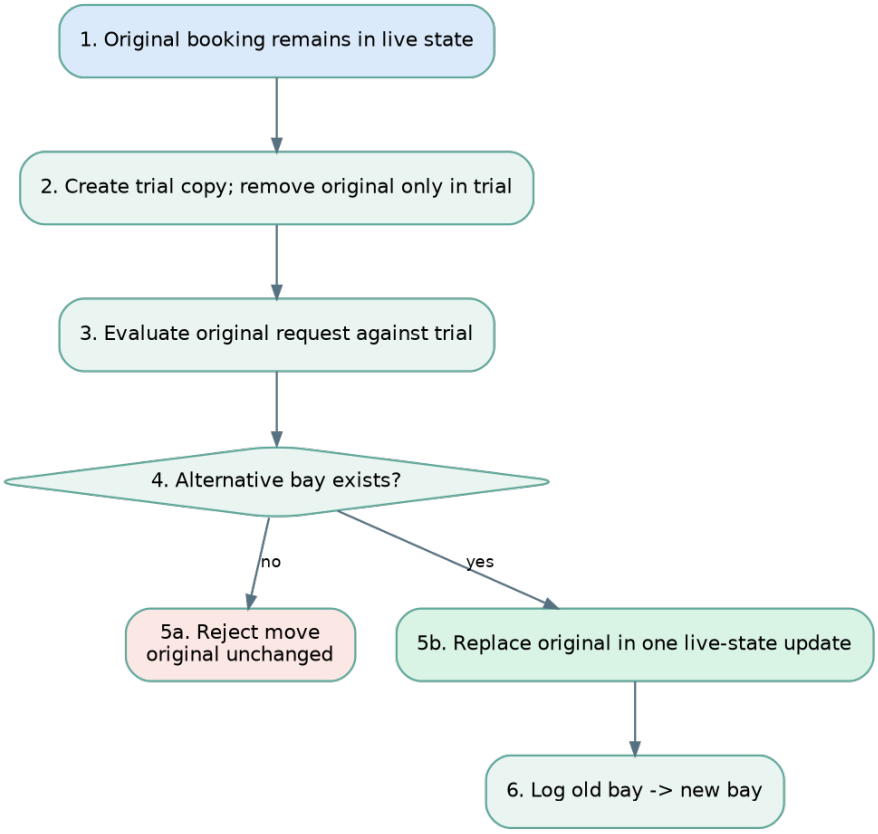


Figure 13. Safe replacement procedure. A failed search leaves the original live booking untouched.

The first version required cancellation before trying a replacement. Version 2 changes that behavior: the original booking is removed only in a temporary copy while alternatives are evaluated. The live collection is replaced only after a different eligible bay is found. This protects against accidental loss of a reservation when the facility is full.

Demonstration

Confirm R1 on G1 and R2 on G2, then close G1. Moving R1 fails because G2 is occupied and the other bays do not meet the request. The original R1 remains on G1 with a closure warning. Cancel R2 and repeat the move; R1 is then replaced with a booking on G2. Both success and rollback are covered by tests.

The action re-evaluates the whole original interval. Moving an already-active booking is therefore conservative: it may reject an alternative that was occupied only during the elapsed portion. A future real-time system should split elapsed and remaining intervals. The current interface disables moves on completed bookings.

7.2 Code structure and executable components

Table 13. Source inventory

Component	Purpose
engine.py	Frame defaults, validation, fact adapter, grounding, indexed inference, ranking and actions
policy.json / frames.json	34 rule schemas and frame definitions; shared input to both runtimes
engine.js	Equivalent browser-side adapter, engines and booking actions
ui.html / ui.js	Responsive interface, map, timeline, controls and guided scenarios
ParkWise_Simulation.html	Self-contained generated simulation; no dependencies to open
app.py	Optional Python local HTTP API and HTML delivery
theory.py / ontology.ttl	Limited taxonomy classification, finite integrity checks and richer DL specification
scenarios.py	16 reproducible scenario fixtures
test_engine.py / test_theory.py	44 unit tests
validate_all.py / test_ui.cjs	Cross-language, baseline-equivalence and DOM integration checks
build_report.py / build_diagrams.py	Editable report content and exact diagram generation

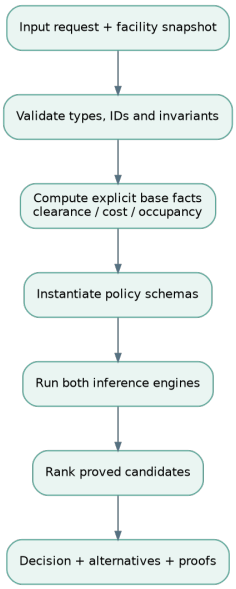


Figure 14. Data flow for one evaluation. All evidence is tied to its evaluated input snapshot.

Appendix A includes the complete core Python implementation. The ZIP also includes every supporting source file and the diagram source graphs, so the report is not the only description of the system.

7.2 Key code: interval, cost and confirmation

Adapter arithmetic

```
def overlaps(a,b):return a['start']<b['end'] and b['start']<a['end']

def estimate(r,b):return math.ceil((r['end']-r['start'])*b['rate']/60)
```

The overlap function is symmetric and covers containment without special cases. The price calculation prorates the hourly rate to the requested duration and rounds upward to an integer. The adapter exposes the results through positive facts such as booking_clear, budget_ok and duration_ok.

Confirmation

```
def commit(state):
    result=evaluate(state)
    if not result['recommended']:raise ValueError('No compatible available bay. Change the request or
    wait for capacity.')
    state['bookings'].append({**state['request'],'bay':result['recommended']});return result
```

The result is recomputed before mutation, so a previous on-screen recommendation cannot force an invalid confirmation. Exactly one chosen bay is saved, even though multiple candidate facts may exist. Duplicate request IDs and existing overlapping bookings are rejected by validation.

Grounding and rule scope

```
for schema in POLICY:
    if schema["scope"] == "request":
        bindings = [{"r": request_id}]
    elif schema["scope"] == "bay":
        bindings = [{"r": request_id, "b": bay["id"]} for bay in bays]
    else:
        bindings = [{"q": booking["id"]} for booking in bookings]
    # Substitute each binding into the schema head and body.
```

This is finite grounding, not variable unification inside the engines. The distinction matters during the viva: the domain rules are parameterized, but both inference implementations operate on concrete atom strings.

7.3 Running the interactive simulation

Recommended launch
Open ParkWise_Simulation.html in Chrome, Edge, Firefox or Safari on a computer. It contains the interface and engine in one file. No Python installation, network connection or public deployment is needed.

Interface areas

Table 14. Interface controls and their purpose

Area	What it demonstrates
New parking request	Vehicle, interval, budget, verification, badge, charging and dimensions
Live parking map	Ten bays; eligibility, occupancy, uncertainty and maintenance status
Bay inspector	Change sensor readings, close/reopen a bay, repair/fault a charger
Simulation clock	Move time manually or play in five-minute increments
Reservation timeline	Visualize half-open intervals and their current lifecycle state
Reasoning workspace	Decision reasons, stepwise forward trace, expandable backward proof and facts
Scenario laboratory	Load any of 16 independently reset fixtures
Guided demo	Eight events covering confirmation, disruption, failed move and recovery
Export evidence	Download current state, last evaluated snapshot, proof and event log

Optional Python route

```
python app.py
# Open http://127.0.0.1:8765
python -m unittest test_engine test_theory -v
python test_http.py
python engine.py
```

The offline simulation does not call the Python API. Both versions are delivered for learning and comparison. On Windows, the launcher prefers the offline HTML so the presentation does not depend on Python being installed.

The interface was exercised through DOM integration tests, not a successful rendered-browser session in this environment. A local visual check on the presentation device remains recommended. No fabricated browser screenshot is included.

8.1 Results: use cases 01-08

UC01 · Normal

Standard car: G1 is nearest eligible bay.

Actual result: G1; candidates: G1, G2.

Reason: G1 satisfies all four H16 branches and has the shortest distance among eligible bays. G2 is a lower-price but longer-walk alternative.

UC02 · Overlap

Partial interval overlap moves the next car to G2.

Actual result: G2; candidates: G2.

Reason: the existing G1 booking overlaps the new 09:30-10:30 request; H17 derives blocked for G1.

UC03 · Adjacent

A departure at 10:00 permits a new arrival at 10:00.

Actual result: G1; candidates: G1, G2.

Reason: the previous end equals the new start, so the strict overlap comparison is false.

UC04 · Ev

EV charging uses E1; E2 has a charger fault.

Actual result: E1; candidates: E1.

Reason: H10 requires a ready charger; E2 fails because charger_fault is explicit.

UC05 · Badge

Badge holder can use A1 and A2; A1 is nearest.

Actual result: A1; candidates: A1, G1, G2, A2.

Reason: H08 permits accessible bays, then normal ranking selects A1.

UC06 · Budget

PKR 85 budget excludes G1 but allows G2.

Actual result: G2; candidates: G2.

Reason: G1 costs PKR 100, exceeding the PKR 85 budget; G2 costs PKR 80.

UC07 · Compact

A compact car with PKR 60 budget fits the upper deck.

Actual result: G3; candidates: G3.

Reason: the 1.6 m vehicle fits G3; its PKR 50 fee fits the PKR 60 ceiling.

UC08 · Sensor

G1 occupancy is unknown, so manual review is required.

Actual result: G2; candidates: G2.

Reason: unknown occupancy causes manual_review rather than assuming the bay is empty.

8.1 Results: use cases 09-16

UC09 · Maintenance

G1 closes after confirmation. Use Move on R1 for safe reallocation.

Actual result: G2; candidates: G2.

R1 remains on closed G1 until moved. The fixture evaluates a new R2 request; use Move on R1 to test replacement.

UC10 · Full

Both compatible general bays are booked; no candidate remains.

Actual result: No candidate; candidates: none.

No candidate is a correct outcome: general capacity is exhausted for this request; reserved-purpose or incompatible bays are not substituted.

UC11 · Oversize

A 2.5 m vehicle exceeds every bay height limit.

Actual result: No candidate; candidates: none.

UC12 · Afterhours

05:00-06:00 is outside the facility schedule.

Actual result: No candidate; candidates: none.

UC13 · Duration

A 30-minute stay with PKR 35 budget uses G4.

Actual result: G4; candidates: G4.

A 30-minute stay costs PKR 30 on G4 and fits its maximum-stay rule. A longer stay is rejected for G4.

UC14 · Motorcycle

Motorcycle category uses M1.

Actual result: M1; candidates: M1.

UC15 · Fault

Both charging units are faulty, so charging is unavailable.

Actual result: No candidate; candidates: none.

UC16 · Disabled

A disabled account cannot receive a candidate.

Actual result: No candidate; candidates: none.

8.1 Verification and evidence

Table 15. Executed checks and their meaning

Check	Result	What it establishes
Python unit tests	44 passed	Expected decisions, lifecycle, validation, atomic move and theory subset
Python vs JavaScript comparison	616 snapshots passed	Same candidates, recommendation, facts, derived atoms, reasons and costs
DOM integration	49 assertions passed	Handlers render correct text/state for scenarios, demo, proof and trace controls
HTTP smoke check	Passed	State, HTML, evaluate, commit, cancellation, closure and reset endpoints
Indexed vs scan closure	Equal at 10 / 100 / 500 bays	Optimization preserves the tested consequence sets

```
Ran 44 tests in 1.636s

OK

PASS: 49 DOM interaction assertions; 16 scenarios; eight-event guided demo; proof/trace and lifecycle
    controls.
```

The 616 comparison cases comprise 600 seeded generated snapshots plus the 16 scenario fixtures. Generated inputs vary vehicle category, verification, accessibility badge, charging demand, dimensions, budget, time and bay closures/uncertainty. Both languages consume the same policy schemas, so parity alone does not prove the policy is correct. Expected-result tests provide a separate check of intended behavior.

Negative and boundary tests

The suite includes adjacent intervals, contained intervals, duplicate IDs, invalid charging requests, malformed time ranges, NaN values, unknown sensors, oversized vehicles, disabled accounts, cyclic rule graphs, duplicate premises and empty-body rules. Reallocation failure checks compare the entire original state before and after the attempt.

DOM tests are programmatic interface tests in jsdom. They do not verify real browser layout, visual clipping, native downloads or browser-specific time-input behavior. Those limitations are reported rather than hidden behind a generic “fully tested” claim.

8.2 Efficiency and scalability

Table 16. Measured inference-only times; median of seven runs

Bays	Facts	Ground rules	Scan (ms)	Indexed (ms)
10	120	238	0.617	0.341
100	1128	2308	5.743	2.184
500	5608	11508	26.924	12.989

These measurements compare bottom-up closure algorithms only. They exclude input validation, grounding, backward proofs and interface rendering. They were recorded on the build environment and are not production latency guarantees. Both algorithms produced the same closure in each measured case.

Why the index helps

The scan baseline revisits every rule on each fixed-point round. The indexed algorithm processes the occurrences of each newly known premise and decrements only the affected counters. For a fixed ground KB, its core work is proportional to stored premise occurrences plus derived heads, apart from set and map operations. It trades additional index memory for fewer repeated rule checks.

Remaining costs

The adapter checks interval conflicts against the booking collection for every bay. Existing-booking integrity validation compares booking pairs. Consequently this prototype is not asymptotically optimized for a large historical database. A production extension should index reservations by bay and time, use interval queries and enforce transactional exclusion constraints.

The backward engine uses a head index but does not implement full tabling. This is adequate for the acyclic policy here. Scalability claims are limited to the measured finite experiments. The recommendation is greedy per request and can consume a bay that a later request needs; global planning is outside scope.

8.2 Discussion: expressiveness and limitations

How the representations cooperate

Frames supply defaults and typed structure; the semantic network explains relationships; FOL states quantified conditions; DL organizes terminology and disjointness; Horn clauses compute policy consequences; application actions change state. Each has a specific role. The project is strongest when these boundaries are visible rather than pretending one reasoner implements every formalism.

Expressiveness versus tractability

Full FOL can express richer quantified relationships than the finite ground engine executes. Definite Horn clauses give a simple monotonic consequence model with explainable proof steps. DL supplies a structured vocabulary and open-world semantics, but the implemented classifier covers only named superclass and disjointness operations. Interval arithmetic, price calculations and ranking remain deterministic application computations.

Material limitations

• Synthetic data and policies: no official parking rules, real sensors, payment service or identity-verification system.

• Same-day integer-minute intervals: no dates, overnight bookings, recurrence or measured travel times.

• Session-only storage: no durable database, authentication, multi-user concurrency or deployment claim.

• Conservative sensor interpretation: current uncertainty blocks even future requests.

• Reallocation checks the whole original interval and replaces records without keeping a separate immutable booking-history table.

• No complete FOL theorem prover or full OWL/DL reasoner; supported subsets are described and tested.

• DOM interaction tests passed, but real-browser visual verification remains to be completed locally.

These limits do not negate the educational objective: the project demonstrates a substantial, inspectable KRR system. They identify the exact work required before extending it into a real operational service.

9. Conclusion and future development

ParkWise turns a parking request into an explainable set of candidate decisions using 34 policy schemas, 10 bays and explicit interval, service and operational facts. Its design differs materially from role-and-permit campus allocation: it models vehicle dimensions, affordability, stay limits, charging faults, occupancy uncertainty, lifecycle changes and safe replacement of reservations.

The implementation demonstrates executable frame inheritance, a limited taxonomy classifier, finite integrity checks, two independent Horn inference strategies and a self-contained interactive simulator. The test evidence establishes consistency across the delivered Python and JavaScript versions for the evaluated scenarios and generated snapshots, while preserving a clear boundary between tested results and unverified deployment claims.

Prioritized future improvements

Table 17. Development roadmap

Priority	Improvement	Reason
1	Visual walkthrough on the presentation device	Verify layout, browser controls and downloaded evidence
2	Persisted booking store with transactional constraints	Support recovery and concurrent users
3	Dates, overnight intervals and effective sensor timestamps	More faithful temporal modelling
4	Remaining-interval reallocation and audit history	Handle active disruptions without rewriting elapsed history
5	Full OWL reasoner integration	Evaluate richer ontology restrictions
6	Global allocation / waitlist policy	Improve utilization across multiple requests

Submission readiness

The technical package covers every section named in the supplied CT-351 report template. The remaining group-specific fields are names, roll numbers, section, instructor, submission date and actual contributions. The supplied CCP assignment brief has also been checked; its unmarked grading sheets are included after the title page. The supplement maps its requirements to evidence. Parking is proposed under the brief’s equivalent-complex-domain option; no separate domain-approval decision is asserted.

10. References and provenance

[1] NED University of Engineering & Technology, Department of Computer Science and IT. *CT-351 Knowledge Representation & Reasoning: Complex Computing Problem Report Template.* User-supplied course document, undated. Used for report organization and required sections.

[2] Hitzler, P., Kroetzsch, M., Parsia, B., Patel-Schneider, P. F., and Rudolph, S. (eds.). *OWL 2 Web Ontology Language Primer, Second Edition.* W3C Recommendation, 11 December 2012. https://www.w3.org/TR/owl2-primer/ (accessed 10 November 2026). Used for ontology terminology, class relationships and restrictions.

[3] Poole, D. L., and Mackworth, A. K. *Artificial Intelligence: Foundations of Computational Agents, Third Edition.* Section 5.3, Propositional Definite Clauses. https://artint.info/3e/html/ArtInt3e.Ch5.S3.html (accessed 10 November 2026). Used for definite-clause semantics and inference background.

[4] NED University of Engineering & Technology. *CT-351 Complex Computing Problem assignment brief and grading rubric.* Four-page user-supplied course document. Governs the requirement mapping and reproduced blank rubric sheets.

Design provenance

Two user-supplied Smart Parking Access & Space Allocation reports were examined to understand the shared domain and the requested level of detail. ParkWise does not reuse their campus-role policy, named participants or claimed fieldwork. Its facility assumptions, code, scenario fixtures, diagrams and recorded tests belong to this delivered implementation.

Evidence files

Table 18. Reproducibility artifacts

File	Contents
test_results.txt	Full Python unit-test transcript
http_results.txt	HTTP smoke-test result
ui_results.txt	DOM integration-test transcript
validation_results.json	Scenario outputs, benchmark numbers and comparison count
parity_cases.json	616 exact comparison snapshots
knowledge.json / horn_kb.txt	Default facts, grounded policy, conclusions, trace and proofs
diagrams/*.dot	Editable sources of all 14 report diagrams

No interviews, real-world performance study or student-authored contribution is fabricated. Complete the participation section with the work actually performed by each member.

11. Team participation and viva preparation

Table 19. Proposed responsibility allocation; not a claim of completed student work

Member / roll number	Suggested responsibility	Evidence to understand
Awaiting details	Domain, requirements, frames and semantic relations	Sections 1-3; frames.json
Awaiting details	FOL/DL specification and integrity reasoning	Section 4; theory.py; ontology.ttl
Awaiting details	Policy, grounding and inference algorithms	Sections 5-6; policy.json; engine.py
Awaiting details	Simulation, scenarios, tests and presentation	Sections 7-8; ui.js; test files

Viva questions and expected understanding

• Why can several candidate facts be true while only one booking is confirmed? Eligibility and assignment are different relations.

• Why is 10:00-11:00 allowed after 09:00-10:00? The end boundary is exclusive.

• Why does sensor_unknown not imply sensor_free? Missing or uncertain evidence is not permission to allocate.

• What is executed as Horn logic and what is computed by the adapter? Know the arithmetic, validation and ranking boundary.

• What does atomic reallocation protect? The original booking survives when no replacement can be proved.

• What does the DL component actually support? Named superclass closure and declared disjointness checks, not full OWL reasoning.

• How is originality demonstrated? A different policy focus, data model, implementation and set of tested scenarios.

Declaration to complete after review

Group representative: _____ Date: _____

Members should review and understand the complete system, complete the real contribution record and follow their course rules for acknowledging assistance. The report currently leaves identity fields open rather than guessing names or assigning unperformed work.

Teacher brief: requirement-to-evidence map

The teacher asks for a multi-paradigm knowledge representation and rule-based reasoning system. ParkWise applies this to a synthetic mixed-use parking facility. The four suggested domains are examples; the brief also permits an equivalent complex domain to be proposed.

Table 20. Submission evidence for the supplied assignment

Required element	Evidence in this submission
Title and grading rubric	Title page and two original, unmarked rubric sheets immediately after it.
Domain summary and challenge	Sections 1-2: timed allocation, dimensions, charging, accessibility, uncertainty and disruption recovery.
Traditional database limitations	The expressivity and tractability supplement explains the distinction without claiming SQL cannot express rules.
Frames, slots, facets, defaults and scripts	Section 3; frames.json and executable inheritance; formal script on the next page.
Semantic network and frame architecture	Section 3 diagrams; editable diagrams/*.dot and rendered PNG files.
FOL and Description Logics	Section 4: quantified constraints, subsumption, membership and disjointness; theory.py scope stated.
Horn knowledge base	Section 5: all 34 policy schemas; horn_kb.txt includes the grounded default KB.
Forward and backward trace sheets	Section 6; full default goal expansion in this supplement; exported knowledge.json.
Execution outputs and code	Sections 7-8, complete core appendix, reproducible tests and standalone HTML.
Team participation	Section 11 provides roles to complete with actual names, roll numbers and work performed.

Additional requested deliverable: animated video

video/ParkWise_Animated_Demo.mp4 is a 2 minute 22 second, 1280 × 720 animation with captions and an original synthesized background soundtrack. Its scenario decisions are computed by engine.py. Vehicle motion is illustrative; this is an animated reconstruction, not a screen recording or live sensor demonstration. The renderer, subtitles and scenario evidence are included for reproduction.

Formal script: reserve and recover a bay

A script represents an expected sequence of domain events. Frames describe entities and defaults; this script connects their roles and state transitions. Its nominal path and exception paths are implemented by the simulator’s guided events and the engine actions, rather than by a generic script interpreter.

Table 21. ParkingReservationAndRecovery script

Field	Specification
Roles	Driver/request operator; facility operator; inference engine; examiner.
Props	Request frame, vehicle attributes, bay frames, booking list, facility status, inference trace.
Entry conditions	Valid request identifier and interval; current bay snapshot and complete booking collection; no existing booking with the same request identifier.
Scene 1: request	Capture verification, vehicle category/dimensions, interval, badge, charging need and budget. Validate facets and request shape.
Scene 2: infer	Ground snapshot facts; forward-chain candidate goals; independently backward-prove each candidate. Reject or explain every unmet condition.
Scene 3: select	Rank eligible bays by distance, then calculated cost, then bay ID. Recommendation alone does not reserve a bay.
Scene 4: commit	Re-evaluate the current state and append a booking only if a candidate remains. Preserve the original input if validation fails.
Scene 5: disruption	Operator marks the assigned bay closed or otherwise unavailable. The existing reservation remains recorded pending action.
Scene 6: recover	Remove the affected booking only in a trial copy; evaluate its original request; exclude the old bay; commit a replacement only when one is proved.
Exception tracks	Invalid input: explain validation failure. No candidate: retain state. Unknown sensor: require review. Failed move: retain old booking. Cancellation: explicitly release its interval.
Exit conditions	A confirmed eligible reservation, an explicitly cancelled booking, or an unchanged state with a recorded explanation of failure.

Concrete execution

The animated demonstration confirms R1 on G1 and R2 on G2, closes G1, demonstrates a failed move, releases R2, then moves R1 to G2. A later request starting exactly when R1 ends demonstrates the half-open interval convention. These transitions are separate from illustrated car movement.

Representation limits and computational trade-offs

Traditional databases and rule-based knowledge

A relational database can store vehicles, bays and reservations efficiently and enforce keys, checks and transactions. SQL joins, recursive queries, views and triggers can also implement many parking policies. The limitation is not an inability to express rules: a plain data schema does not by itself supply inheritance defaults, ontology classifications or a reusable explanation of a derived decision. Embedding each policy in application queries can scatter its meaning and make policy changes harder to audit.

ParkWise separates facts from a shared Horn policy and records rule identifiers and supporting premises. This makes eligibility explanations inspectable. It does not replace a production database: the delivered simulator keeps session state in memory and lacks persistent transactions across concurrent users. A future system should combine database integrity and persistence with the reasoning layer.

Propositional representation

A ground atom such as candidate(R1,G1) behaves propositionally in the executable engine. With a finite request and bay snapshot, inference can be bounded and reproduced. Pure propositional notation cannot state one quantified rule for every possible request and bay; each applicable instance must be enumerated. Adding entities therefore increases the grounded knowledge base. General propositional inference can be expensive, but positive definite Horn clauses admit efficient fixed-point algorithms.

First-Order Logic and Description Logics

FOL expresses relations, universal constraints and existential statements compactly. For example, a universal no-overlap requirement covers all booking pairs without hand-writing each pair. Unrestricted FOL validity is only semidecidable: a general prover need not terminate on every input. Richer notation alone therefore cannot guarantee a practical decision procedure. Description Logic offers restricted ontology languages with explicit class semantics; computational costs depend on the selected constructors and reasoning task.

The deliberate executable boundary

ParkWise grounds a finite, function-free, positive Horn policy for a supplied snapshot. Arithmetic, interval comparisons, validation and ranking are computed by explicit adapters. Indexed forward chaining processes newly enabled rules; backward chaining checks a chosen goal through its supporting premises. Unknown sensor evidence is not interpreted as free. Snapshot booking-clear facts depend on the supplied booking collection being complete.

The FOL specification is not a general theorem prover. The DL component implements named-class superclass closure and declared disjointness checks, not a complete OWL reasoner. These boundaries preserve predictable execution while the report’s richer models document the intended domain semantics. Benchmark results in Section 8 describe measured inference times only, not general complexity guarantees or live deployment performance.

Backward chaining: complete default goal trace

Input: initial() snapshot, request R1, 09:00-10:00, standard car. The following preorder expansion is generated directly from the independent backward engine. Indentation denotes subgoals. Every leaf is an explicit snapshot fact; every internal conjunction succeeds only after all its children succeed. All 31 listed calls succeed.

```
CALL / GOAL                                SOURCE RESULT
01  candidate(R1,G1)              H16   TRUE
02    admissible(R1)                H15   TRUE
03      authorized(R1)                H01   TRUE
04        verified(R1)                  fact  TRUE
05        account_enabled(R1)           fact  TRUE
06      schedulable(R1)               H02   TRUE
07        request(R1)                   fact  TRUE
08        valid_window(R1)              fact  TRUE
09        inside_hours(R1)              fact  TRUE
10    compatible(R1,G1)             H13   TRUE
11      size_fit(R1,G1)               H05   TRUE
12        car_sized(R1)                 H03   TRUE
13          car(R1)                       fact  TRUE
14        car_space(G1)                 fact  TRUE
15      access_fit(R1,G1)             H07   TRUE
16        request(R1)                   fact  TRUE
17        unrestricted(G1)              fact  TRUE
18      service_fit(R1,G1)            H09   TRUE
19        no_charge(R1)                 fact  TRUE
20        noncharging(G1)               fact  TRUE
21      dimension_fit(R1,G1)          H11   TRUE
22        height_ok(R1,G1)              fact  TRUE
23        width_ok(R1,G1)               fact  TRUE
24    available(R1,G1)              H14   TRUE
25      bay(G1)                       fact  TRUE
26      operational(G1)               fact  TRUE
27      sensor_free(G1)               fact  TRUE
28      booking_clear(R1,G1)          fact  TRUE
29    commercial_fit(R1,G1)         H12   TRUE
30      budget_ok(R1,G1)              fact  TRUE
31      duration_ok(R1,G1)            fact  TRUE
```

Return and interpretation

After the final budget and duration leaves succeed, H12 returns commercial_fit(R1,G1). All four children of H16 are now true, so candidate(R1,G1) returns TRUE. This confirms eligibility; deterministic ranking selects G1, and commit performs the separate reservation action. The forward trace in Section 6 reaches the same candidate through data-driven rule firing.

For contrast, the default proof for A1 fails at its accessibility branch because badge(R1) is absent. A failed proof means not derivable from this snapshot; it is not a universal claim that the bay can never be allocated. Inspect the complete successful and failed trees in knowledge.json or the interactive proof viewer.

Appendix A. Complete Python core

Exact engine.py source, line-numbered for reference. Continuation lines preserve the original code text; use the source file to run or edit it.

```
001 """ParkWise v2: finite policy grounding, indexed Horn engines and atomic booking actions."""
002 from dataclasses import dataclass, asdict
003 from copy import deepcopy
004 from collections import defaultdict, deque
005 from pathlib import Path
006 import json, math, re
007 ROOT=Path(__file__).resolve().parent
008 POLICY=json.loads((ROOT/'policy.json').read_text())
009 FRAME_CATALOG=json.loads((ROOT/'frames.json').read_text())
010 @dataclass(frozen=True)
011 class Rule:
012     name:str
013     head:str
014     body:tuple
015
016 def atom(p,*args):return p+'('+','.join(map(str,args))+')'
017 def frame_defaults(name,trail=()):
018     if name in trail:raise ValueError('Cyclic frame inheritance.')
019     f=FRAME_CATALOG[name];base=frame_defaults(f['parent'],trail+(name,)) if 'parent' in f else {}
020     return {**base,**f.get('slots',{})}
021 def make_bay(id,kind,distance,**kw):return
    {**frame_defaults('Bay'),**dict(id=id,kind=kind,distance=distance),**kw}
022 BAYS=[make_bay('G1','general',20),make_bay('G2','general',40,rate=80),
023  make_bay('A1','accessible',5),make_bay('E1','charging',25,rate=180,charger='ready',zone='Charging
     court',max_minutes=180),
024  make_bay('M1','motorcycle',15,rate=40),make_bay('G3','general',65,rate=50,zone='Upper
    deck',max_height=1.7),
025  make_bay('G4','general',30,rate=60,max_minutes=30),make_bay('E2','charging',45,rate=160,charger='
    fault',zone='Charging court'),
026  make_bay('A2','accessible',55,rate=90,zone='Upper
    deck'),make_bay('G5','general',50,rate=70,sensor='occupied',zone='Upper deck')]
027 def request(id='R1',vehicle='car',start=540,end=600,verified=True,badge=False,charge=False,**kw):
028     return {**frame_defaults('Request'),**dict(id=id,vehicle=vehicle,start=start,end=end,verified=
    verified,badge=badge,charge=charge),**kw}
029 def initial():return {'bays':deepcopy(BAYS),'bookings':[],'request':request(),'clock':540}
030 def overlaps(a,b):return a['start']<b['end'] and b['start']<a['end']
031 def estimate(r,b):return math.ceil((r['end']-r['start'])*b['rate']/60)
032 def valid_id(x):return isinstance(x,str) and re.fullmatch(r'[A-Za-z0-9]{1,30}',x)
033 def number(x,minimum=0):return type(x) in (int,float) and math.isfinite(x) and x>=minimum
034
035 def validate(state):
036     bays=state['bays']; bookings=state['bookings'];r=state['request'];ids=[b['id'] for b in bays]
037     if not all(valid_id(i) for i in ids) or len(set(ids))!=len(ids):raise ValueError('Bay IDs must
     be unique alphanumeric values.')
038     if type(state.get('clock',540)) is not int or not 0<=state.get('clock',540)<=1440:raise
    ValueError('Invalid simulation clock.')
039     for b in bays:
040         if b['kind'] not in ('general','accessible','charging','motorcycle') or type(b['open']) is
     not bool:raise ValueError('Invalid bay frame.')
041         if b['sensor'] not in ('free','occupied','unknown') or b['charger'] not in
    ('none','ready','fault'):raise ValueError('Invalid sensor or charger status.')
042         if not number(b['distance']) or type(b['rate']) is not int or b['rate']<0:raise
    ValueError('Invalid distance or hourly rate.')
043         if not number(b['max_height'],.1) or not number(b['max_width'],.1) or
    type(b['max_minutes']) is not int or b['max_minutes']<1:raise ValueError('Invalid bay limits.')
044     for q in [r]+bookings:
045         if not valid_id(q['id']):raise ValueError('Use 1-30 letters and digits for request IDs.')
046         if type(q['start']) is not int or type(q['end']) is not int or not
    0<=q['start']<q['end']<=1440:raise ValueError('Use a valid same-day time interval.')
047         if q['vehicle'] not in ('car','ev','motorcycle'):raise ValueError('Invalid vehicle type.')
048         if any(type(q[k]) is not bool for k in ('verified','badge','charge','enabled')):raise
    ValueError('Flags must be boolean.')
049         if q['charge'] and q['vehicle']!='ev':raise ValueError('Only an electric vehicle may
    request charging.')
```

Appendix A. Complete Python core (continued)

Exact engine.py source, line-numbered for reference. Continuation lines preserve the original code text; use the source file to run or edit it.

```
050         if not number(q['height'],.1) or not number(q['width'],.1) or type(q['budget']) is not int
     or q['budget']<0:raise ValueError('Invalid dimensions or budget.')
051     if len({q['id'] for q in bookings})!=len(bookings):raise ValueError('Duplicate booking IDs.')
052     if any(q['id']==r['id'] for q in bookings):raise ValueError('This request ID already has a
    booking.')
053     for i,q in enumerate(bookings):
054         if q['bay'] not in ids:raise ValueError('Booking references an unknown bay.')
055         if any(p['bay']==q['bay'] and overlaps(p,q) for p in bookings[:i]):raise
    ValueError('Existing bookings overlap.')
056
057 def forward(facts,rules):
058     known=set(facts);trace=[];waiting=defaultdict(list);remaining=[];queue=deque(sorted(facts))
059     def derive(r):
060         if r.head not in known:
061             known.add(r.head);queue.append(r.head);trace.append({'rule':r.name,'fact':r.head,'beca
    use':list(r.body)})
062     for i,r in enumerate(rules):
063         body=set(r.body);remaining.append(len(body))
064         for b in body:waiting[b].append(i)
065     for i,r in enumerate(rules):
066         if not remaining[i]:derive(r)
067     while queue:
068         for i in waiting[queue.popleft()]:
069             remaining[i]-=1
070             if remaining[i]==0:derive(rules[i])
071     return known,trace
072
073 def backward(goal,facts,rules,path=(),index=None):
074     if index is None:
075         index=defaultdict(list)
076         for r in rules:index[r.head].append(r)
077     if goal in facts:return {'goal':goal,'proved':True,'source':'fact','children':[]}
078     if goal in path:return {'goal':goal,'proved':False,'source':'cycle','children':[]}
079     attempts=[]
080     for r in index.get(goal,[]):
081         children=[backward(b,facts,rules,path+(goal,),index) for b in r.body]
082         node={'goal':goal,'proved':all(c['proved'] for c in
    children),'source':r.name,'children':children}
083         if node['proved']:return node
084         attempts.append(node)
085     return {'goal':goal,'proved':False,'source':'unproved','children':attempts}
086
087 def knowledge(state):
088     validate(state);r=state['request'];rid=r['id'];facts=set();rules=[];failures={};costs={}
089     def f(p,*args):facts.add(atom(p,*args))
090     f('request',rid);f(r['vehicle'],rid);f('valid_window',rid)
091     for yes,no,ok in [('verified','unverified',r['verified']),('account_enabled','account_disabled
    ',r['enabled']),('badge','no_badge',r['badge']),('charge','no_charge',r['charge']),('inside_hours'
    ,'outside_hours',360<=r['start']<r['end']<=1320)]:f(yes if ok else no,rid)
092     for b in state['bays']:
093         bid=b['id'];kind=b['kind'];f('bay',bid);f('motorcycle_space' if kind=='motorcycle' else
    'car_space',bid)
094         f('accessible' if kind=='accessible' else 'unrestricted',bid);f('charging' if
    kind=='charging' else 'noncharging',bid)
095         f('operational' if b['open'] else 'closed',bid);f('sensor_'+b['sensor'],bid)
096         if kind=='charging':f('charger_ready' if b['charger']=='ready' else 'charger_fault',bid)
097         clear=not any(q['bay']==bid and overlaps(q,r) for q in state['bookings'])
098         f('booking_clear' if clear else 'booking_conflict',rid,bid)
099
    height=r['height']<=b['max_height'];width=r['width']<=b['max_width'];costs[bid]=estimate(r,b)
100         if height:f('height_ok',rid,bid)
101         if width:f('width_ok',rid,bid)
102         if not (height and width):f('dimension_exceeded',rid,bid)
103         budget=costs[bid]<=r['budget'];duration=r['end']-r['start']<=b['max_minutes']
```

Appendix A. Complete Python core (continued)

Exact engine.py source, line-numbered for reference. Continuation lines preserve the original code text; use the source file to run or edit it.

```
104         f('budget_ok' if budget else 'over_budget',rid,bid);f('duration_ok' if duration else
    'over_duration',rid,bid)
105         checks={'Vehicle
    category':(r['vehicle']=='motorcycle')==(kind=='motorcycle'),'Accessibility
    badge':kind!='accessible' or r['badge'],
106          'Charging service':(kind!='charging' or (r['vehicle']=='ev' and r['charge'] and
    b['charger']=='ready')) and (not r['charge'] or kind=='charging'),
107          'Vehicle clearance':height and width,'Budget':budget,'Maximum stay':duration,'Overlapping
     booking':clear,'Maintenance':b['open'],'Sensor status':b['sensor']=='free',
108          'Verified account':r['verified'],'Enabled account':r['enabled'],'Opening
    hours':360<=r['start']<r['end']<=1320}
109         failures[bid]=[key for key,ok in checks.items() if not ok]
110     for q in state['bookings']:
111         clock=state.get('clock',540);f('booking_future' if clock<q['start'] else 'booking_current'
     if clock<q['end'] else 'booking_past',q['id'])
112     for rule in POLICY:
113         bindings=([{'r':rid}] if rule['scope']=='request' else [{'r':rid,'b':b['id']} for b in
    state['bays']] if rule['scope']=='bay' else [{'q':q['id']} for q in state['bookings']])
114         for bind in
    bindings:rules.append(Rule(rule['id'],rule['head'].format(**bind),tuple(x.format(**bind) for x in
    rule['body'])))
115     return facts,rules,failures,costs
116
117 def evaluate(state):
118     facts,rules,failures,costs=knowledge(state);known,trace=forward(facts,rules);rid=state['reques
    t']['id'];index=defaultdict(list)
119     for r in rules:index[r.head].append(r)
120     candidates=sorted([b for b in state['bays'] if atom('candidate',rid,b['id']) in
    known],key=lambda b:(b['distance'],costs[b['id']],b['id']))
121     proofs={b['id']:backward(atom('candidate',rid,b['id']),facts,rules,index=index) for b in
    state['bays']}
122     if not all(proofs[b['id']]['proved']==(atom('candidate',rid,b['id']) in known) for b in
    state['bays']):raise RuntimeError('Inference disagreement.')
123     return {'candidates':[b['id'] for b in candidates],'recommended':candidates[0]['id'] if
    candidates else None,'facts':sorted(facts),'derived':sorted(known-
    facts),'trace':trace,'proofs':proofs,'failures':failures,'costs':costs,'rules':[asdict(r) for r in
     rules]}
124 def commit(state):
125     result=evaluate(state)
126     if not result['recommended']:raise ValueError('No compatible available bay. Change the request
     or wait for capacity.')
127     state['bookings'].append({**state['request'],'bay':result['recommended']});return result
128
129 def cancel(state,rid):
130     before=len(state['bookings']);state['bookings']=[q for q in state['bookings'] if
    q['id']!=rid];return before!=len(state['bookings'])
131 def reallocate(state,rid):
132     original=next((q for q in state['bookings'] if q['id']==rid),None)
133     if original is None:raise ValueError('Booking not found.')
134     trial=deepcopy(state);trial['bookings']=[q for q in trial['bookings'] if
    q['id']!=rid];trial['request']={k:v for k,v in original.items() if k!='bay'}
135     result=evaluate(trial);options=[b for b in result['candidates'] if b!=original['bay']]
136     if not options:raise ValueError('No replacement available; original booking retained.')
137     replacement={**trial['request'],'bay':options[0]};state['bookings']=[replacement if
    q['id']==rid else q for q in state['bookings']]
138     return {'from':original['bay'],'to':options[0],'result':result}
139 if __name__=='__main__':print(json.dumps(evaluate(initial()),indent=2))
```

Appendix B. Reproduction and presentation checklist

Quick presentation route

```
1. Extract ParkWise_Project.zip.
2. Open ParkWise_Simulation.html in a desktop browser.
3. Load Standard arrival and inspect both inference views.
4. Start the eight-event guided demo; click Next event.
5. Inspect the failed move, then the successful recovery.
6. Export evidence before closing the session.
```

Developer validation route

```
python -m unittest test_engine test_theory -v
python test_http.py
python build_simulation.py
python validate_all.py    # requires Node.js
npm install jsdom
node test_ui.cjs
python build_diagrams.py  # requires Graphviz
python build_report.py    # requires reportlab
```

Complete submission pack

• PDF report and its editable Python build source.

• Standalone simulation and full Python/JavaScript source.

• Frame catalogue, ontology and complete Horn KB export.

• Sixteen scenario fixtures and exact test-result files.

• Fourteen diagrams with editable Graphviz sources.

Before presenting, check the standalone page on the actual computer, confirm the browser allows the evidence download, complete the group-specific report fields and rehearse the explanations. The simulation can be demonstrated without an internet connection. The optional development dependencies are only needed for rebuilding or testing, not for opening the HTML file.